

Amazon Kinesis

Table of Contents

1. Streaming data.....	4
2 Architecture of Kinesis Data Streams	5
3. Step-by-step instructions to install and configure the AWS CLI on Windows	6
4. Kinesis Data Streams operations using the AWS CLI	7
5. Create and manage Kinesis data streams.....	14
6. Produce or write data to kinesis data stream	22
6.1 Produce a stream using JAVA SDK	22
6.1.1 Generate random samples(Order Genarator)	22
6.1.2 create a product class	25
6.1.3 Create Order Class.....	27
6.1.4 Order Item Class:	31
6.1.5 PUT Record to Kinesis.....	33
6.2 Produce a stream using Python SDK	39
6.2.1 Generate random samples(Order Genarator)	39
6.2.2 PUT Record to Kinesis.....	41
6.3 JAVA:Producing Records In Batches to KDS	45
6.4 Python: Producing Records In Batches to KDS.....	47
6.5 Java: Producing Records with Java and the Kinesis Producer Library	48
7. Consume or read data from kinesis data stream.....	54
7.1 Java : Consuming Records from KDS.....	55
7.2 Python: Consuming Records from KDS	62
7.3 Problems of GetRecords based SDK Consumers	65
7.4 Java : Consuming Records with Java and the Kinesis Client Library	67
7.5 Consume Records using AWS Lambda	83
8. Amazon Data Firehose.....	86
8.1 Create a Firehose stream	87
8.2 Transform source data in Amazon Data Firehose	104
8.3 Partition streaming data in Amazon Data Firehose	106
8.4 Understand data delivery	110
9. Deliver data to Apache Iceberg Tables with Amazon Data Firehose	111
10. Amazon Kinesis Data Analytics Using Apace Flink Zeplin Notebook.....	116
10.1 Introduction to Flink SQL	117
10.2 Streaming vs Batch Processing in Flink SQL	118

10.3 Time Semantics in Flink SQL	122
10.4 Watermarks in Flink SQL.....	123
10.5 Windowing Functions in Flink SQL — Real-Time Examples	124
10.6 Table Definitions (DDL) in Flink SQL.....	128
10.7 Streaming Joins in Flink SQL.....	131
10.8 Aggregations in Flink SQL	133
10.9 Sinks and Outputs in Flink SQL.....	134
10.10 Fault Tolerance & State Management in Flink SQL	136
10.11 User-Defined Functions (UDFs) in Flink SQL.....	138
10.12 Pattern Detection with Flink SQL (CEP)	139
10.13 SQL over Change Data Capture (CDC)	140
10.14 CloudWatch Integration with Apache Flink	142

1. Streaming data

Streaming data is data that is emitted at high volume in a continuous, incremental manner with the goal of low-latency processing. Organizations have thousands of data sources that typically simultaneously emit messages, records, or data ranging in size from a few bytes to several megabytes (MB). Streaming data includes location, event, and sensor data that companies use for real-time analytics and visibility into many aspects of their business. For example, companies can track changes in public sentiment on their brands and products by continuously analyzing clickstream and customer posts from social media streams then responding promptly as needed.

Here are some more examples of streaming data.

Data analysis

Applications process data streams to produce reports and perform actions in response, such as emitting alarms when key measures exceed certain thresholds. More sophisticated stream processing applications extract deeper insights by applying machine learning algorithms to business and customer activity data.

IoT applications

Internet of Things (IoT) devices are another use case for streaming data. Sensors in vehicles, industrial equipment, and farm machinery send data to a streaming application. The application monitors performance, detects potential defects in advance, and automatically places a spare part order, preventing equipment downtime.

Financial analysis

Financial institutions use stream data to track real-time changes in the stock market, compute value at risk, and automatically rebalance portfolios based on stock price movements. Another financial use case is fraud detection of credit card transactions using real-time inferencing against streaming transaction data.

Real-time recommendations

Real estate applications track geolocation data from consumers' mobile devices and make real-time recommendations of properties to visit. Similarly, advertising, food, retail, and consumer applications can integrate real-time recommendations to give more value to customers.

Service guarantees

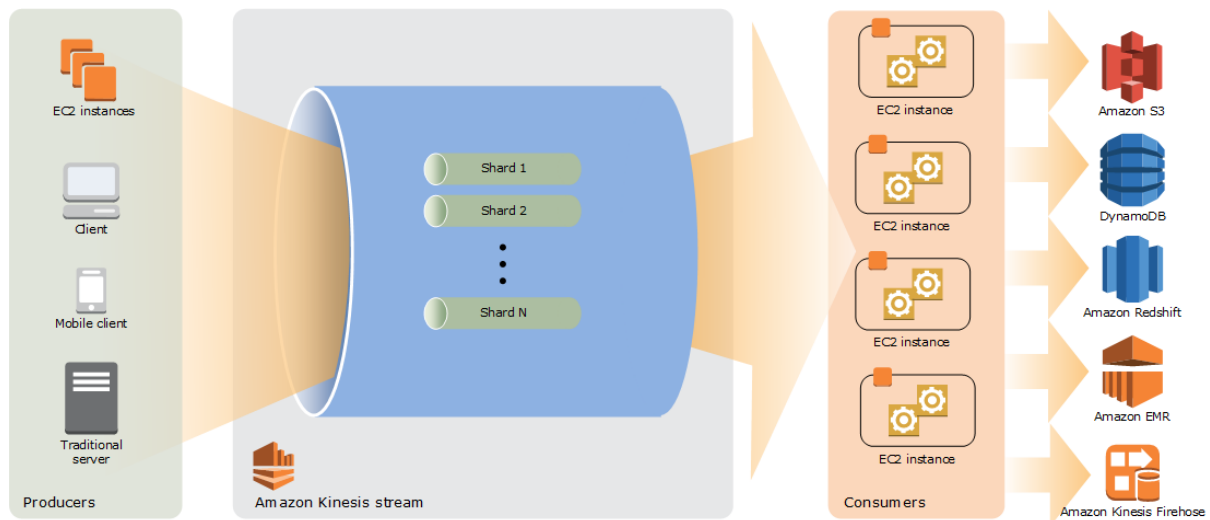
You can implement data stream processing to track and maintain service levels in applications and equipment. For example, a solar power company has to maintain power throughput for its customers or pay penalties. It implements a streaming data application that monitors all panels in the field and schedules service in real time. Thus, it can minimize each panel's periods of low throughput and the associated penalty payouts.

Media and gaming

Media publishers stream billions of clickstream records from their online properties, aggregate and enrich the data with user demographic information, and optimize the content placement. This helps publishers deliver a better, more relevant experience to audiences. Similarly, online

gaming companies use event stream processing to analyze player-game interactions and offer dynamic experiences to engage players.

2 Architecture of Kinesis Data Streams



The above diagram illustrates the high-level architecture of Kinesis Data Streams. The *producers* continually push data to Kinesis Data Streams, and the *consumers* process the data in real time. Consumers (such as a custom application running on Amazon EC2 or an Amazon Data Firehose delivery stream) can store their results using an AWS service such as Amazon DynamoDB, Amazon Redshift, or Amazon S3.

◆ Core Concepts

- **Data Stream:** A collection of **shards**, each holding a sequence of **data records**.
- **Shard:** A unit of capacity in a stream. Each shard supports:
 - **5 reads/sec** (max 2 MB/sec)
 - **1,000 writes/sec** (max 1 MB/sec)

◆ Data Record

- Contains:
 - **Sequence Number** (unique per partition key within shard)
 - **Partition Key** (used to determine shard)
 - **Data Blob** (up to 1 MB, immutable byte sequence)

◆ Capacity Modes

- **On-Demand Mode:**
 - Auto-managed shards
 - Charged based on actual throughput

- **Provisioned Mode:**
 - Manually set number of shards
 - Charged hourly per shard
-

◆ Retention Period

- Default: **24 hours**
 - Can be increased up to **365 days (8760 hours)**
 - Extra charges apply for >24 hours
-

◆ Roles

- **Producer:** Sends data to the stream (e.g., web server logs)
 - **Consumer:** Retrieves and processes data from the stream
-

◆ Kinesis Data Streams Application

- Runs on **EC2 instances**
 - Types:
 - **Shared Fan-Out Consumer**
 - **Enhanced Fan-Out Consumer**
 - Can send output to other streams or AWS services
-

◆ Partition Key

- Unicode string (max 256 characters)
 - Used to map data to shards via MD5 hash
-

3. Step-by-step instructions to install and configure the AWS CLI on Windows

Step1: Install or update the AWS CLI

To update your current installation of AWS CLI on Windows, download a new installer each time you update to overwrite previous versions. AWS CLI is updated regularly. To see when the latest version was released, see the [AWS CLI version 2 Changelog](#) on *GitHub*.

1. Download and run the AWS CLI MSI installer for Windows (64-bit):
<https://awscli.amazonaws.com/AWSCLIV2.msi>

Step 2: Install AWS CLI

- Double-click the downloaded `.msi` file.
- Follow the installation wizard:
 - Accept the license agreement.
 - Choose the installation location (default is fine).
 - Click **Install**.
- Once installed, you can verify by opening **Command Prompt** and typing
- `aws --version`

Step 3: Configure AWS CLI

- Open **Command Prompt** and run:
 - `aws configure`
- Create user in aws and give administration access and copy the access key and secret key from that user.
- You'll be prompted to enter:
 1. **AWS Access Key ID**
 2. **AWS Secret Access Key**
 3. **Default region name** (e.g., `us-east-1`, `ap-south-1`)
 4. **Default output format** (e.g., `json`, `text`, `table`)

The above steps same for configure in vs code.

4. Kinesis Data Streams operations using the AWS CLI

AVAILABLE COMMANDS

- o `add-tags-to-stream`
- o `create-stream`
- o `delete-stream`
- o `describe-stream`
- o `get-records`
- o `get-shard-iterator`
- o `help`
- o `list-streams`
- o `list-tags-for-stream`

- o merge-shards
- o put-record
- o put-records
- o remove-tags-from-stream
- o split-shard
- o wait

Here's the **AWS CLI syntax** for each of the listed Kinesis stream commands:

◆ add-tags-to-stream

Adds or updates tags for a stream.

```
aws kinesis add-tags-to-stream \  
  --stream-name <stream-name> \  
  --tags Key1=Value1,Key2=Value2
```

◆ create-stream

Creates a new Kinesis data stream.

```
aws kinesis create-stream \  
  --stream-name <stream-name> \  
  --shard-count <number-of-shards>
```

◆ delete-stream

Deletes a stream.

```
aws kinesis delete-stream \  
  --stream-name <stream-name> \  
  [--enforce-consumer-deletion]
```

◆ describe-stream

Describes the specified stream.

```
aws kinesis describe-stream-summary \
```

```
--stream-name <stream-name>
```

```
"Shards": [  
  {  
    "ShardId": "shardId-000000000000",  
    "HashKeyRange": {  
      "StartingHashKey": "0",  
      "EndingHashKey": "340282366920938463463374607431768211455"  
    },  
    "SequenceNumberRange": {  
      "StartingSequenceNumber":  
"49590338271490256608559692538361571095921575989136588898"  
    }  
  }  
]  
}
```

◆ get-records

Gets data records from a shard.

```
aws kinesis get-records \
```

```
--shard-iterator <iterator-token> \
```

```
[--limit <number-of-records>]
```

```
{  
  "Records": [  
    {  
      "SequenceNumber":  
"49640610668614886128711708916060667677544601327287074818",  
      "Data": "dGVzdDE=",  
      "PartitionKey": "key1"  
    },  
    {  
      "SequenceNumber":  
"49640610668614886128711708916060667677544601327287074819",  
      "Data": "dGVzdDI=",  
      "PartitionKey": "key2"  
    }  
  ],  
  "NextShardIterator": "AAAAAAAAAAEXAMPLENEXT"  
}
```

◆ `get-shard-iterator`

Gets a shard iterator for reading data.

```
aws kinesis get-shard-iterator \  
  --stream-name <stream-name> \  
  --shard-id <shard-id> \  
  --shard-iterator-type <type> \  
  [--starting-sequence-number <sequence-number>]
```

Shard Iterator Types: AT_SEQUENCE_NUMBER, AFTER_SEQUENCE_NUMBER, TRIM_HORIZON, LATEST, AT_TIMESTAMP

```
{  
  "ShardIterator": "AAAAAAAAAAEXAMPLE"  
}
```

◆ `help`

Displays help information.

```
aws kinesis help
```

◆ `list-streams`

Lists all streams.

```
aws kinesis list-streams
```

```
{  
  "StreamNames": [  
    "samplestream",  
    "logstream",  
    "eventstream"  
  ]  
}
```

◆ `list-tags-for-stream`

Lists tags associated with a stream.

```
aws kinesis list-tags-for-stream \  
  --stream-name <stream-name> \  
  --shard-id <shard-id> \  
  --shard-iterator-type <type> \  
  [--starting-sequence-number <sequence-number>]
```

`--stream-name <stream-name>`

◆ `merge-shards`

Merges two adjacent shards.

```
aws kinesis merge-shards \  
  --stream-name <stream-name> \  
  --shard-to-merge <shard-id> \  
  --adjacent-shard-to-merge <shard-id>
```

◆ `put-record`

Writes a single data record.

```
aws kinesis put-record \  
  --stream-name <stream-name> \  
  --partition-key <key> \  
  --data <base64-encoded-data>  
  
{  
  "ShardId": "shardId-0000000000000",  
  "SequenceNumber":  
"49640610668614886128711708916060667677544601327287074818"  
}
```

Data is **Base64-encoded** data.

For decode

```
import base64  
print(base64.b64decode("dGVzdCBtZXNzYWdlIGJhc2U2NA==").decode)
```

◆ `put-records`

Writes multiple records in one call.

```
aws kinesis put-records \  
  --stream-name <stream-name> \  
  --records '[{"Data": "<base64-data>", "PartitionKey": "<key>"}, {...}]'  
  
{  
  "FailedRecordCount": 0,
```

```
"Records": [  
  {  
    "SequenceNumber":  
"49640610668614886128711708916060667677544601327287074818",  
    "ShardId": "shardId-000000000000"  
  },  
  {  
    "SequenceNumber":  
"49640610668614886128711708916060667677544601327287074819",  
    "ShardId": "shardId-0000000000001"  
  }  
]  
}
```

◆ `remove-tags-from-stream`

Removes tags from a stream.

```
aws kinesis remove-tags-from-stream \  
  --stream-name <stream-name> \  
  --tag-keys Key1 Key2
```

◆ `split-shard`

Splits a shard into two.

```
aws kinesis split-shard \  
  --stream-name <stream-name> \  
  --shard-to-split <shard-id> \  
  --new-starting-hash-key <hash-key>
```

◆ `wait`

Waits until a stream reaches a desired state.

```
aws kinesis wait stream-exists \  
  --stream-name <stream-name>
```

Other waiters: `stream-exists`, `stream-not-exists`

- ◆ Increase Retention Period

Command:

```
aws kinesis increase-stream-retention-period \  
--stream-name <stream-name> \  
--retention-period-hours <hours>
```

Shard ID:

A **shard** is a unit of capacity in a Kinesis data stream. Each shard can ingest and process a certain amount of data per second.

- **Shard ID** is the unique identifier for a shard in a stream.
- It looks like: `shardId-000000000000`, `shardId-000000000001`, etc.

Example:

Suppose you have a stream called `UserActivityStream` with 2 shards:

- `shardId-000000000000`
- `shardId-000000000001`

Sequence Number:

A **sequence number** is a unique identifier assigned to each record within a shard. It helps maintain the order of records.

- It's a string like:
`49545115243490985018280067714973144582180062593244200961`
- It increases over time as new records are added.

Shard Iterator:

A **shard iterator** is a pointer used to read data from a shard. It tells Kinesis where to start reading records.

There are different types of iterators:

- `TRIM_HORIZON`: Start from the oldest record.
- `LATEST`: Start from the newest record.
- `AT_SEQUENCE_NUMBER`: Start from a specific sequence number.
- `AFTER_SEQUENCE_NUMBER`: Start just after a specific sequence number.

5. Create and manage Kinesis data streams

Choose the data stream capacity mode:

A capacity mode determines how the capacity of a data stream is managed and how you are charged for the usage of your data stream. In Amazon Kinesis Data Streams, you can choose between an **on-demand** mode and a **provisioned** mode for your data streams.

- **On-demand** - data streams with an on-demand mode require no capacity planning and automatically scale to handle gigabytes of write and read throughput per minute. With the on-demand mode, Kinesis Data Streams automatically manages the shards in order to provide the necessary throughput.
- **Provisioned** - for the data streams with a provisioned mode, you must specify the number of shards for the data stream. The total capacity of a data stream is the sum of the capacities of its shards. You can increase or decrease the number of shards in a data stream as needed.

You can use Kinesis Data Streams `PutRecord` and `PutRecords` APIs to write data into your data streams in both on-demand and provisioned capacity modes. To retrieve data, both capacity modes support default consumers that use the `GetRecords` API and Enhanced Fan-Out (EFO) consumers that use the `SubscribeToShard` API.

All Kinesis Data Streams capabilities, including retention mode, encryption, monitoring metrics, and others, are supported for both the on-demand and provisioned modes. Kinesis Data Streams provides the high durability and availability in both the on-demand and provisioned capacity modes.

On-demand mode:

◆ Scalability & Performance

- **No capacity planning needed** – automatically scales to handle gigabytes of read/write throughput per minute.
- **Ideal for unpredictable or highly variable traffic** – no need to provision for peak capacity.
- **Supports billions of records per day** with **low latency** and **no operational overhead**.

◆ Cost Model

- **Pay-per-use:** Charged based on **GBs of data written and read**.

- No need to specify expected throughput in advance.

◆ Automatic Scaling Behavior

- **Doubles peak write throughput** observed in the **last 30 days**.
- Example: If peak was 40 MB/s, it can scale up to 80 MB/s.
- If a new peak of 50 MB/s is sustained, it can scale to 100 MB/s.
- **Write throttling** may occur if traffic **doubles within 15 minutes** – retry throttled requests.

◆ Read Capacity

- **Scales proportionally** with write throughput.
- **GetRecords API**: Read capacity is **at least 2x the write throughput**.
- **Recommendation**: Use **one consumer** with GetRecords for recovery scenarios.

◆ Enhanced Fan-Out (EFO)

- Use **EFO** for **multiple consumer applications**.
- Supports **up to 20 consumers** per stream.
- Each consumer gets **dedicated throughput** via `SubscribeToShard` API.

Throughput:

Throughput refers to the **amount of data** that can be **processed in a given amount of time**. In the context of **Kinesis Data Streams**, it specifically means:

- **Write Throughput**: How much data (in MB per second) your producers can send to the stream.
- **Read Throughput**: How much data (in MB per second) your consumers can read from the stream.

Example:

Let's say your Kinesis stream has:

- **Write throughput** of **2 MB/s** per shard.
- **Read throughput** of **4 MB/s** per shard (because read is 2× write in on-demand mode).

Enhanced Fan-Out (EFO):

Enhanced Fan-Out (EFO) is a feature of **Amazon Kinesis Data Streams** that allows **multiple consumer applications** to **simultaneously and independently** read data from the same data stream **without sharing throughput**.

Here are the key points:

- **Dedicated throughput per consumer:** Each EFO consumer gets its **own 2 MB/second** of read throughput per shard, independent of other consumers.
- **Supports up to 20 consumers** per data stream.
- Uses the **SubscribeToShard API** instead of the traditional `GetRecords` API.
- **Low latency:** EFO delivers records to consumers with **latency as low as 70 milliseconds**.
- Ideal for **real-time processing** and **multiple parallel applications** (e.g., analytics, monitoring, ML).
- **No need to manage read throughput sharing** between consumers.
- **Higher cost** compared to standard consumers, but provides better performance and scalability.

Provisioned mode :

With provisioned mode, after you create the data stream, you can dynamically scale your shard capacity up or down using the AWS Management Console or the [UpdateShardCount](#) API. You can make updates while there is a Kinesis Data Streams producer or consumer application writing to or reading data from the stream.

The provisioned mode is suited for predictable traffic with capacity requirements that are easy to forecast. You can use the provisioned mode if you want fine-grained control over how data is distributed across shards.

With the provisioned mode, you must specify the number of shards for the data stream. To determine the size of a data stream with the provisioned mode, you need the following input values:

- The average size of the data record written to the stream in kilobytes (KB), rounded up to the nearest 1 KB (`average_data_size_in_KB`).
- The number of data records written to and read from the stream per second (`records_per_second`).

- The number of consumers, which are Kinesis Data Streams applications that consume data concurrently and independently from the stream (`number_of_consumers`).
- The incoming write bandwidth in KB (`incoming_write_bandwidth_in_KB`), which is equal to the `average_data_size_in_KB` multiplied by the `records_per_second`.
- The outgoing read bandwidth in KB (`outgoing_read_bandwidth_in_KB`), which is equal to the `incoming_write_bandwidth_in_KB` multiplied by the `number_of_consumers`.

You can calculate the number of shards (`number_of_shards`) that your stream needs by using the input values in the following formula.

```
number_of_shards = ceiling(max(incoming_write_bandwidth_in_KiB/1024,
outgoing_read_bandwidth_in_KiB/2048))
```

Switch between capacity modes:

- You can switch between **on-demand** and **provisioned** capacity modes.
- Each data stream can be switched **up to 2 times in 24 hours**.
- Switching modes **does not disrupt** read/write operations.
- During the switch, the stream status is set to **Updating**.
- You must wait until the stream is **Active** again to make further changes.

Switching from Provisioned → On-Demand:

- The stream **keeps its current shard count** initially.
- After switching, **Kinesis automatically scales** the shard count based on write throughput.

Switching from On-Demand → Provisioned:

- The stream also **keeps its current shard count** initially.
- After switching, **you are responsible** for managing and scaling the shard count manually.

Create a stream using the AWS Management Console:

To create a data stream using the console

1. Sign in to the AWS Management Console and open the Kinesis console at <https://console.aws.amazon.com/kinesis>.
2. In the navigation bar, expand the Region selector and choose a Region.
3. Choose Create data stream.

4. On the Create Kinesis stream page, enter a name for your data stream and then choose either the On-demand or Provisioned capacity mode. The On-demand mode is selected by default. For more information, see [Choose the data stream capacity mode](#).

With the On-demand mode, you can then choose Create Kinesis stream to create your data stream. With the Provisioned mode, you must then specify the number of shards you need, and then choose Create Kinesis stream.

On the Kinesis streams page, your stream's Status is Creating while the stream is being created. When the stream is ready to use, the Status changes to Active.

5. Choose the name of your stream. The Stream Details page displays a summary of your stream configuration, along with monitoring information.

Update a stream:

To update a data stream using the console

1. Open the Amazon Kinesis console at <https://console.aws.amazon.com/kinesis/>.
2. In the navigation bar, expand the Region selector and choose a Region.
3. Choose the name of your stream in the list. The Stream Details page displays a summary of your stream configuration and monitoring information.
4. To switch between on-demand and provisioned capacity modes for a data stream, choose Edit capacity mode in the Configuration tab. For more information, see [Choose the data stream capacity mode](#).
5. For a data stream with the provisioned mode, to edit the number of shards, choose Edit provisioned shards in the Configuration tab, and then enter a new shard count.
6. To edit the data retention period, choose Edit in the Data retention period section, and then enter a new data retention period.
7. If you have enabled custom metrics on your account, choose Edit in the Shard level metrics section, and then specify metrics for your stream. For more information, see [Monitor the Amazon Kinesis Data Streams service with Amazon CloudWatch](#).

Reshard a stream:

- Resharding allows you to adjust the number of shards in a stream based on data flow changes.
- It is considered an advanced operation.
- There are two types of resharding:
 - Shard Split: One shard is split into two shards → increases capacity and cost.
 - Shard Merge: Two shards are merged into one shard → reduces capacity and cost.
- Resharding is pairwise:
 - You can only split one shard into two at a time.
 - You can only merge two shards into one at a time.
- The original shards are called parent shards.
- The resulting shards are called child shards.

Split a shard:

◆ What is Shard Splitting?

- **Shard splitting** is a type of **resharding** operation.
- It **divides one shard into two**, increasing the stream's capacity.
- You must **manually specify** how to split the **hash key range** of the parent shard.

◆ How Hash Keys Work:

- Every data record in Kinesis has a **partition key**.
- Kinesis computes an **MD5 hash** of this partition key → this becomes the **hash key**.
- Each shard handles a **range of hash keys** (from a starting to an ending value).

◆ How to Split a Shard:

1. **Get the hash key range** of the shard:

```
shard.getHashKeyRange().getStartingHashKey();
```

```
shard.getHashKeyRange().getEndingHashKey();
```

2. **Choose a split point** (usually the midpoint between start and end hash keys):

```
BigInteger startingHashKey = new  
BigInteger(shard.getHashKeyRange().getStartingHashKey());
```

```
BigInteger endingHashKey = new  
BigInteger(shard.getHashKeyRange().getEndingHashKey());
```

```
String newStartingHashKey = startingHashKey.add(endingHashKey).divide(new  
BigInteger("2")).toString();
```

3. **Create and configure the split request:**

```
SplitShardRequest splitShardRequest = new SplitShardRequest();
```

```
splitShardRequest.setStreamName(myStreamName);
```

```
splitShardRequest.setShardToSplit(shard.getShardId());
```

```
splitShardRequest.setNewStartingHashKey(newStartingHashKey);
```

4. **Execute the split:** `client.splitShard(splitShardRequest);`

◆ Result of the Split:

- The **parent shard** is replaced by **two child shards**.
- One child gets the **lower half** of the hash key range.
- The other gets the **upper half**.
- This increases the number of shards → **more capacity**, but also **higher cost**.

Merge two shards:

◆ What is Shard Merging?

- A **shard merge** operation **combines two adjacent shards** into **one shard**.
- The resulting **child shard** handles the **combined hash key range** of both parent shards.

◆ Shard Adjacency (Important Rule)

- Two shards can only be merged if they are **adjacent**.
- **Adjacent** means their hash key ranges form a **continuous range with no gaps**.

✓ Example (can merge):

- Shard A: 276 . . . 381
- Shard B: 382 . . . 454

- Combined: 276...454 → **contiguous**

✗ Example (cannot merge):

- Shard A: 276...381
- Shard B: 455...560
- Missing range: 382...454 → **not contiguous**

◆ Shard States

- Only **OPEN shards** can be merged.
- A shard is **OPEN** if its `EndingSequenceNumber` is `null`.

```
if (null == shard.getSequenceNumberRange().getEndingSequenceNumber()) {  
  
    // Shard is OPEN and can be considered for merging  
  
}
```

◆ How to Merge Shards (Java Example)

1. **Create a merge request:**

```
MergeShardsRequest mergeShardsRequest = new MergeShardsRequest();  
  
mergeShardsRequest.setStreamName(myStreamName);  
  
mergeShardsRequest.setShardToMerge(shard1.getShardId());  
  
mergeShardsRequest.setAdjacentShardToMerge(shard2.getShardId());
```

2. **Execute the merge:** `client.mergeShards(mergeShardsRequest);`

6. Produce or write data to kinesis data stream

6.1 Produce a stream using JAVA SDK

6.1.1 Generate random samples(Order Genarator)

This Java class `OrderGenerator` is part of a package named `com.thecodinginterface.kinesis`. It is designed to **generate random orders** consisting of products, sellers, and customers. Let's break it down line by line and explain what each part does:

Package Declaration

```
package com.thecodinginterface.kinesis;
```

- Declares that this class belongs to the `com.thecodinginterface.kinesis` package.

Imports

```
import java.util.ArrayList;
```

```
import java.util.Arrays;
```

```
import java.util.List;
```

```
import java.util.UUID;
```

```
import java.util.concurrent.ThreadLocalRandom;
```

- These are Java utility classes used in the code:
 - `ArrayList`, `Arrays`, `List`: For working with collections.
 - `UUID`: For generating unique order IDs.
 - `ThreadLocalRandom`: For generating random numbers in a thread-safe way.

Class Declaration

```
public class OrderGenerator {
```

- Defines a public class named `OrderGenerator`.

Static Arrays

```
static final Product[] products = {
```

```
    new Product("A0001", "Hair Comb", 2.99),
```

```

new Product("A0002", "Toothbrush", 5.99),
new Product("A0003", "Dental Floss", 0.99),
new Product("A0004", "Hand Soap", 1.99)
};

```

- Defines a static array of `Product` objects. Each product has:
 - A code (e.g., "A0001")
 - A name (e.g., "Hair Comb")
 - A price (e.g., 2.99)

```

static final String[] sellerIDs = { "abc", "xyz", "jkq", "wrp" };
``

```

- Array of seller IDs.

```

static final String[] customerIDs = {
    "C1000", "C1001", ..., "C1019"
};
``

```

- Array of customer IDs.

Method: `makeOrder()`

```

public static Order makeOrder() {
``

```

- This method generates a random `Order` object.

Generate Unique Order ID

```

var orderID = UUID.randomUUID().toString();
``

```

- Creates a unique order ID using `UUID`.

Random Generator

```

var tlr = ThreadLocalRandom.current();
``

```

- Gets the current thread-local random generator.

Random Seller and Customer

```
var sellerID = sellerIDs[tlr.nextInt(sellerIDs.length)];
```

```
var customerID = customerIDs[tlr.nextInt(customerIDs.length)];
```

- Randomly selects a seller and a customer from the arrays.

Copy Product List

```
List<Product> availableProducts = new ArrayList<>(Arrays.asList(products));
```

```
``
```

- Converts the static array of products into a mutable list so items can be removed during selection.

Create Order Items

```
List<OrderItem> orderItems = new ArrayList<>();
```

```
int nProducts = tlr.nextInt(1, products.length);
```

- Initializes an empty list of `OrderItems`.
- Randomly decides how many products to include in the order (between 1 and total number of products).

Loop to Add Products

```
for (int i = 0; i < nProducts; i++) {
```

```
    Product p = availableProducts.remove(tlr.nextInt(availableProducts.size()));
```

```
    var qty = tlr.nextInt(1,10);
```

```
    orderItems.add(new OrderItem(p.getName(), p.getCode(), qty, p.getPrice()));
```

```
}
```

- For each product to be added:
 - Randomly selects and removes a product from the list.
 - Randomly selects a quantity between 1 and 9.
 - Creates an `OrderItem` with product details and adds it to the list.

Return the Order

```
return new Order(orderItems, customerID, orderID, sellerID);
```

- Constructs and returns a new `Order` object with:
 - The list of items
 - Customer ID
 - Order ID
 - Seller ID

6.1.2 create a product class

This `Product` class is used in the earlier `OrderGenerator` code to represent individual products in an order.

`Product` class: This class defines the structure of product.

Code:

```
package com.thecodinginterface.kinesis;  
``
```

This declares the package name. It helps organize the class and group it with related classes in the same module or project.

```
public class Product {
```

Defines a public class named `Product`. This class models a product with a code, name, and price.

```
private String code;  
private String name;  
private double price;
```

These are **private fields** (variables) that store the product's:

- `code`: a unique identifier (e.g., "A0001")
- `name`: the product name (e.g., "Hair Comb")
- `price`: the cost of the product (e.g., 2.99)

```
public Product(){}
```

This is a **default constructor**. It allows creating a `Product` object without setting any values initially.

```
public Product(String code, String name, double price) {  
    this.code = code;  
    this.name = name;  
    this.price = price;  
}
```

This is a **parameterized constructor**. It allows creating a `Product` object with specific values for `code`, `name`, and `price`.

```
public String getCode() {  
    return code;  
}
```

Getter method for `code`. It returns the product's code.

```
public void setCode(String code) {  
    this.code = code;  
}
```

Setter method for `code`. It allows updating the product's code.

```
public String getName() {  
    return name;  
}
```

Getter method for `name`. It returns the product's name.

```
public void setName(String name) {  
    this.name = name;  
}
```

Setter method for `name`. It allows updating the product's name.

```
public double getPrice() {  
    return price;  
}
```

Getter method for `price`. It returns the product's price.

```
public void setPrice(double price) {  
    this.price = price;  
}
```

Setter method for `price`. It allows updating the product's price.

6.1.3 Create Order Class

This class represents a **complete order**. An order contains:

- A list of items (`OrderItem`)
- A customer ID
- An order ID
- A seller ID

It is used in the `OrderGenerator` class to create and return a full order.

```
package com.thecodinginterface.kinesis;
```

Declares the package name. This groups the class with related classes like `Product`, `OrderItem`, and `OrderGenerator`.

```
import com.fasterxml.jackson.annotation.JsonProperty;
import com.google.common.base.Objects;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
```

Imports:

- `JsonProperty`: Used to define custom JSON field names when converting the object to JSON.
- `Objects`: From Google Guava, used for comparing and hashing objects.
- `ArrayList`, `Arrays`, `List`: For working with collections of items.

```
@JsonProperty("order_items")
```

```
private List<OrderItem> orderItems = new ArrayList<>();
```

Defines a list to hold all items in the order. Annotated with `@JsonProperty` so when converted to JSON, the field name will be "order_items".

```
@JsonProperty("customer_id")
```

```
private String customerID;
```

Stores the ID of the customer who placed the order.

```
@JsonProperty("order_id")
```

```
private String orderID;
```

Stores the unique ID of the order.

```
@JsonProperty("seller_id")
```

```
private String sellerID;
```

Stores the ID of the seller who is fulfilling the order.

```
public Order() {}
```

Default constructor — allows creating an empty `Order` object.

```
public Order(List<OrderItem> orderItems, String customerID, String orderID, String sellerID) {  
    this.orderItems = orderItems;  
    this.customerID = customerID;  
    this.orderID = orderID;  
    this.sellerID = sellerID;  
}
```

Parameterized constructor — allows creating an `Order` object with all details provided.

```
public List<OrderItem> getOrderItems() { return orderItems; }  
public void setOrderItems(List<OrderItem> orderItems) { this.orderItems = orderItems; }
```

Getter and setter for the list of order items.

```
public String getCustomerID() { return customerID; }  
public void setCustomerID(String customerID) { this.customerID = customerID; }  
..
```

Getter and setter for customer ID.

```
public String getOrderID() { return orderID; }  
public void setOrderID(String orderID) { this.orderID = orderID; }
```

Getter and setter for order ID.

```
public String getSellerID() { return sellerID; }  
public void setSellerID(String sellerID) { this.sellerID = sellerID; }
```

Getter and setter for seller ID.

@Override

```
public String toString() {  
    return String.format("Order{orderId='%s', customerId='%s', sellerID='%s', orderItems=%s}",  
        orderId, customerId, sellerID, Arrays.toString(orderItems.toArray()));  
}
```

Overrides the default `toString()` method to return a readable string representation of the order.

@Override

```
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
    Order order = (Order) o;  
    return Objects.equal(orderItems, order.orderItems) &&  
        Objects.equal(customerID, order.customerID) &&  
        Objects.equal(orderID, order.orderID) &&  
        Objects.equal(sellerID, order.sellerID);  
}
```

Overrides default `equals()` method to compare two `Order` objects based on their content.

@Override

```
public int hashCode() {  
    return Objects.hashCode(orderItems, customerId, orderID, sellerID);  
}  
``
```

Overrides default `hashCode()` method to generate a hash based on the order's content. Useful when storing orders in hash-based collections like `HashMap`.

- In `OrderGenerator`, after selecting products and creating `OrderItem` objects, the code creates an `Order` object using this class:

```
return new Order(orderItems, customerID, orderID, sellerID);
```

6.1.4 Order Item Class:

```
package com.thecodinginterface.kinesis;

import com.fasterxml.jackson.annotation.JsonProperty;
import com.google.common.base.Objects;

public class OrderItem {

    @JsonProperty("product_name")
    private String productName;

    @JsonProperty("product_code")
    private String productCode;

    @JsonProperty("product_quantity")
    private int productQuantity;

    @JsonProperty("product_price")
    private double productPrice;

    public OrderItem() {}

    public OrderItem(String productName, String productCode, int
productQuantity, double productPrice) {
        this.productName = productName;
        this.productCode = productCode;
        this.productQuantity = productQuantity;
        this.productPrice = productPrice;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public String getProductCode() {
        return productCode;
    }
}
```

```

public void setProductCode(String productCode) {
    this.productCode = productCode;
}

public int getProductQuantity() {
    return productQuantity;
}

public void setProductQuantity(int productQuantity) {
    this.productQuantity = productQuantity;
}

public double getProductPrice() {
    return productPrice;
}

public void setProductPrice(double productPrice) {
    this.productPrice = productPrice;
}

@Override
public String toString() {
    return String.format("OrderItem{productName='%s', productCode='%s',
productQuantity=%d, productPrice=%.2f}",
        productName, productCode, productQuantity, productPrice);
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    OrderItem orderItem = (OrderItem) o;
    return productQuantity == orderItem.productQuantity &&
        Double.compare(orderItem.productPrice, productPrice) == 0 &&
        Objects.equal(productName, orderItem.productName) &&
        Objects.equal(productCode, orderItem.productCode);
}

@Override
public int hashCode() {
    return Objects.hashCode(productName, productCode, productQuantity,
productPrice);
}
}

```


6.1.5 PUT Record to Kinesis

This is a **Java producer application** that sends one order at a time to an **Amazon Kinesis Data Stream** using the `PutRecord` API. It uses the `OrderGenerator` class to create fake orders and sends them to Kinesis.

```
package com.thecodinginterface.kinesis;
```

Declares the package name. This groups the class with others like `Order`, `Product`, etc.

```
import com.fasterxml.jackson.core.JsonProcessingException;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Imports Jackson classes for converting Java objects (like `Order`) into JSON format.

```
import org.apache.log4j.LogManager;
```

```
import org.apache.log4j.Logger;
```

Imports Log4j classes for logging messages to the console or a file.

```
import software.amazon.awssdk.core.SdkBytes;
```

```
import software.amazon.awssdk.services.kinesis.KinesisClient;
```

```
import software.amazon.awssdk.services.kinesis.model.KinesisException;
```

```
import software.amazon.awssdk.services.kinesis.model.PutRecordRequest;
```

```
import software.amazon.awssdk.services.kinesis.model.PutRecordResponse;
```

Imports AWS SDK classes for working with Kinesis:

- `KinesisClient`: used to send data to Kinesis.
- `PutRecordRequest`: defines the data to send.
- `PutRecordResponse`: contains the result of the send operation.
- `SdkBytes`: wraps the data in a format Kinesis understands.

```
public class PutRecordApp {
```

Defines the main class for the producer application.

```
static final Logger logger = LogManager.getLogger(PutRecordApp.class);
```

Creates a logger to print messages for debugging or tracking.

```
static final ObjectMapper objMapper = new ObjectMapper();
```

Creates a Jackson `ObjectMapper` to convert Java objects to JSON.

```
public static void main(String[] args) {
```

Main method — the entry point of the application.

```
logger.info("Starting PutRecord Producer");
```

Logs a message indicating the producer has started.

```
String streamName = args[0];
```

Reads the stream name from command-line arguments. This is the name of the Kinesis stream to send data to.

```
var client = KinesisClient.builder().build();
```

Creates a Kinesis client to interact with AWS Kinesis.

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
```

```
    logger.info("Shutting down program");
```

```
    client.close();
```

```
}, "producer-shutdown"));
```

Adds a shutdown hook — when the program is stopped, this code runs to cleanly close the Kinesis client.

```
while (true) {
```

Starts an infinite loop to continuously send data to Kinesis.

```
var order = OrderGenerator.makeOrder();
```

```
logger.info(String.format("Generated %s", order));
```

Generates a fake order using the `OrderGenerator` class and logs it.

```
try {
```

```
    var putRequest = PutRecordRequest.builder()
```

```
        .partitionKey(order.getOrderID())
```

```
        .streamName(streamName)
```

```
        .data(SdkBytes.fromByteArray(objMapper.writeValueAsBytes(order)))
```

```
        .build();
```

Inside the `try` block:

- Builds a `PutRecordRequest`:
 - `partitionKey`: uses the order ID to distribute records across shards.
 - `streamName`: the name of the Kinesis stream.
 - `data`: converts the `Order` object to JSON and wraps it in `SdkBytes`.
-

```
PutRecordResponse response = client.putRecord(putRequest);
```

```
logger.info(String.format("Produced Record %s to Shard %s", response.sequenceNumber(),  
response.shardId()));
```

Sends the record to Kinesis and logs the response (sequence number and shard ID).

```
} catch(JsonProcessingException e) {
```

```

        logger.error(String.format("Failed to serialize %s", order), e);
    } catch (KinesisException e) {
        logger.error(String.format("Failed to produce %s", order), e);
    }

```

Handles exceptions:

- If JSON conversion fails.
- If sending to Kinesis fails.

```

try {
    Thread.sleep(300);
} catch (InterruptedException e) {
    e.printStackTrace();
}

```

Adds a delay of 300 milliseconds between each record to slow down the output for demonstration purposes.

-
- **Product**: Defines the items in the order.
 - **OrderItem**: Represents each item in the order.
 - **Order**: Represents the full order.
 - **OrderGenerator**: Creates random orders using the above classes.
 - **PutRecordApp**: Sends those orders to Kinesis as JSON records.

```
log4j.rootLogger=INFO, console, file
```

```
log4j.appender.console=org.apache.log4j.ConsoleAppender
```

```
log4j.appender.console.layout=org.apache.log4j.PatternLayout
```

```
log4j.appender.console.layout.ConversionPattern=[%t] %-5p %c %x - %m%n
```

Sets the format of the log messages:

- **%t**: Thread name
- **%p**: Log level (INFO, ERROR, etc.)
- **%c**: Logger name (usually the class name)
- **%x**: Nested diagnostic context (optional)
- **%m**: The actual log message
- **%n**: New line

This is for file logging

log4j.appender.file=org.apache.log4j.RollingFileAppender

log4j.appender.file.layout=org.apache.log4j.PatternLayout

log4j.appender.file.layout.ConversionPattern=[%t] %-5p %c %x - %m%n

log4j.appender.file.File=kinesis.log

place .pom file for build configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.thecodinginterface.kinesis</groupId>
  <artifactId>aws-sdk-kinesis-putrecord</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>2.16.29</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

```

    <dependency>
      <groupId>log4j</groupId>
      <artifactId>log4j</artifactId>
      <version>1.2.17</version>
    </dependency>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.11.3</version>
    </dependency>
    <dependency>
      <groupId>software.amazon.kinesis</groupId>
      <artifactId>amazon-kinesis-client</artifactId>
      <version>2.2.9</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>aws-kinesis-putrecord</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-assembly-plugin</artifactId>
        <version>3.3.0</version>
        <executions>
          <execution>
            <id>make-assembly</id>
            <phase>package</phase>
            <goals>
              <goal>single</goal>
            </goals>
          </execution>
        </executions>
        <configuration>
          <descriptorRefs>
            <descriptorRef>jar-with-dependencies</descriptorRef>
          </descriptorRefs>
          <archive>
            <index>true</index>
            <manifest>
              <addClasspath>true</addClasspath>
              <mainClass>com.thecodinginterface.kinesis.PutRecordApp</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>

```

```
</build>
</project>
```

Steps:

1. So we need to run this script either vs code or IntelliJ.
2. First in terminal we need to configure our aws account.
3. Second step to create aws kinesis stream.(aws kinesis create-stream --stream-name any_name --shard-count number)
4. Run the mvn clean package used to compile, test, and package your Java project.
5. The target file is important after run that command we need to run the dependencies files to generate data into kinesis.
-jar target/aws-kinesis-putrecord-jar-with-dependencies.jar <your-stream-name>
6. After running this command automatically data written to kinesis.

6.2 Produce a stream using Python SDK

6.2.1 Generate random samples(Order Genarator)

```
import copy
import random

from uuid import uuid4

from collections import namedtuple

Product = namedtuple('Product', ['code', 'name', 'price'])

products = [
    Product('A0001', 'Hair Comb', 2.99),
    Product('A0002', 'Toothbrush', 5.99),
    Product('A0003', 'Dental Floss', 0.99),
    Product('A0004', 'Hand Soap', 1.99)
]

seller_ids = ['abc', 'xyz', 'jkq', 'wrp']
customer_ids = [
```

```

    "C1000", "C1001", "C1002", "C1003", "C1004", "C1005", "C1006", "C1007",
    "C1008", "C1009",

    "C1010", "C1011", "C1012", "C1013", "C1014", "C1015", "C1016", "C1017",
    "C1018", "C1019",
]

def make_order():
    order_id = str(uuid4())

    seller_id = random.choice(seller_ids)

    customer_id = random.choice(customer_ids)

    order_items = []

    available_products = copy.copy(products)
    n_products = random.randint(1, len(products))
    for _ in range(n_products):
        product = random.choice(available_products)
        available_products.remove(product)
        qty = random.randint(1, 10)

        order_items.append({
            'product_name': product.name,
            'product_code': product.code,
            'product_quantity': qty,
            'product_price': product.price
        })

    order = {
        'customer_id': customer_id,
        'seller_id': seller_id,
        'order_id': order_id,
        'order_items': order_items
    }

    return order

```



```
Product = namedtuple('Product', ['code', 'name', 'price'])

products = [
    Product('A0001', 'Hair Comb', 2.99),
    Product('A0002', 'Toothbrush', 5.99),
    Product('A0003', 'Dental Floss', 0.99),
    Product('A0004', 'Hand Soap', 1.99)
]
```

What this line does means in java we create the class for product in one java code that in python we used namedtuple.

If you just need a class to store data without methods, `namedtuple` is faster and uses less memory than a full class.

This creates a class-like structure that behaves like a tuple but allows access to fields by name which is immutable.

Use it when:

- You need a simple, immutable data structure.
- You want readable field access.
- You don't need methods or inheritance.

6.2.2 PUT Record to Kinesis

```
"""
Producer app writing single Order records at a time to a Kinesis Data Stream
using
the PutRecord API of the Python SDK.

The globally unique Order ID of each record is used as the partition key which
ensures
order records will be equally distributed across the shard of the stream.
"""

import json
import logging
import sys
import time
```

```

import boto3

from order_generator import make_order

logging.basicConfig(
    format='%(asctime)s %(name)-12s %(levelname)-8s %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    level=logging.INFO,
    handlers=[
        logging.FileHandler("producer.log"),
        logging.StreamHandler(sys.stdout)
    ]
)

def main(args):
    logging.info('Starting PutRecord Producer')

    stream_name = args[1]

    kinesis = boto3.client('kinesis')

    while True:
        # Generate fake order data
        order = make_order()
        logging.info(f'Generated {order}')

        try:
            # execute single PutRecord request
            response = kinesis.put_record(StreamName=stream_name,
                                           Data=json.dumps(order).encode('utf-
8'),
                                           PartitionKey=order['order_id'])

```

```

        logging.info(f"Produced Record {response['SequenceNumber']} to
Shard {response['ShardId']}")

    except Exception as e:
        logging.error({
            'message': 'Error producing record',
            'error': str(e),
            'record': order
        })

    # introduce artificial delay for demonstration and
    # visual tracking of logging
    time.sleep(0.3)

if __name__ == '__main__':
    main(sys.argv)

```

```
available_products = copy.copy(products)
```

uses the `copy` module to create a **shallow copy** of the `products` list.

You might use `copy.copy()` when:

- You want to modify the list (e.g., add/remove items) without affecting the original.
- But you **don't need to clone** the individual items inside the list.

A **shallow copy** creates a new object (in this case, a new list), but **does not copy the inner objects**. Instead, it just copies references to them.

So after this line:

- `available_products` is a **new list**.
- The elements inside it (the `Product` namedtuples) are the **same objects** as in the original `products` list.

```
logging.basicConfig(
```

```
    format='%(asctime)s %(name)-12s %(levelname)-8s %(message)s',
```

```
    datefmt='%Y-%m-%d %H:%M:%S',
```

```

level=logging.INFO,
handlers=[
    logging.FileHandler("producer.log"),
    logging.StreamHandler(sys.stdout)
]
)

format='...'

```

Defines how each log message will look. The placeholders are:

- `%(asctime)s`: Timestamp of the log entry.
- `%(name)-12s`: Logger name, left-aligned in 12-character space.
- `%(levelname)-8s`: Log level (e.g., INFO, ERROR), left-aligned in 8-character space.
- `%(message)s`: The actual log message.

```
datefmt='%Y-%m-%d %H:%M:%S'
```

Specifies the format of the timestamp.

```
level=logging.INFO
```

Sets the minimum severity level to log. Lower levels like `DEBUG` will be ignored.

```
handlers=[...]
```

Specifies where to send the log output:

- `FileHandler("producer.log")`: Writes logs to a file.
- `StreamHandler(sys.stdout)`: Prints logs to the console.

Example:

2025-10-17 12:20:00 Producer INFO Message sent successfully.

```

response = kinesis.put_record(
    StreamName=stream_name,
    Data=json.dumps(order).encode('utf-8'),
    PartitionKey=order['order_id']
)

Data=json.dumps(order).encode('utf-8')

```

- Converts the `order` dictionary to a JSON string using `json.dumps()`.
- Then encodes it to bytes using UTF-8, which is required by Kinesis.

This response variable holds the following details

```
'ShardId': 'shardId-000000000001',
'SequenceNumber': '49590338271490256608559692538361571095921575989136588898',
'ResponseMetadata': {
  'RequestId': '12345678-example',
  'HTTPStatusCode': 200,
  'HTTPHeaders': {
    'x-amzn-requestid': '12345678-example',
    'content-type': 'application/x-amz-json-1.1',
    'content-length': '123',
    'date': 'Fri, 17 Oct 2025 06:52:00 GMT'
  },
  'RetryAttempts': 0
}
}
```

Steps:

1. So we need to run this script either vs code or IntelliJ.
2. First in terminal we need to configure our aws account.
3. Second step to create aws kinesis stream.(aws kinesis create-stream --stream-name any_name --shard-count number)
4. Run the file using the command (python file_name.py kinesis_stream_name)
5. After running this command automatically data written to kinesis.

6.3 JAVA:Producing Records In Batches to KDS

```
Map<PutRecordsRequestEntry, Order> requestEntries = new
LinkedHashMap<>();
while(true) {
    // Generate fake order item data
    var order = OrderGenerator.makeOrder();
    logger.info(String.format("Generated %s", order));

    try {
        // Construct PutRecord request entries
        var entry = PutRecordsRequestEntry.builder()
            .partitionKey(order.getOrderID())
```

```

        .data(SdkBytes.fromByteArray(objMapper.writeValueAsBytes(order)))
        .build();
        requestEntries.put(entry, order);
    } catch (JsonProcessingException e) {
        logger.error(String.format("Failed to serialize %s", order),
e);
    }

    // if there are enough request entries batched up then send them
off to Kinesis
    if (requestEntries.size() > 20) {
        List<PutRecordsRequestEntry> entries = new
ArrayList<>(requestEntries.keySet());
        // construct batched PutRecords request
        var putRequest = PutRecordsRequest.builder()
            .streamName(streamName)
            .records(entries)
            .build();

        PutRecordsResponse response = null;
        try {
            // execute batched PutRecords request
            response = client.putRecords(putRequest);
        } catch (KinesisException e) {
            logger.error("Failed executing PutRecords request", e);
            continue;
        }

        // process the responses
        int i = 0;

        for (PutRecordsResultEntry resultEntry : response.records()) {
            PutRecordsRequestEntry requestEntry = entries.get(i++);
            Order o = requestEntries.get(requestEntry);
            if (resultEntry.errorCode() != null) {
                logger.error(String.format("Failed to produce %s",
o));
            } else {
                logger.info(String.format("Produced %s sequence %s to
Shard %s", o, resultEntry.sequenceNumber(), resultEntry.shardId()));
            }
        }
        requestEntries.clear();
    }
}

```

Every code and steps is same but in putRecordApp code is only different.

This code:

1. Collects multiple orders.
2. Converts each order into a Kinesis-compatible format (`PutRecordsRequestEntry`).
3. Stores them in a map (`requestEntries`) for tracking.
4. Once the map has more than 20 entries, it sends them all at once to Kinesis using `PutRecords`.
5. Then it checks the response for each record and logs whether it was successfully sent or failed.

6.4 Python: Producing Records In Batches to KDS

```
# list for batching up order records to be sent to Kinesis via PutRecords API
order_records = []
while True:
    order = make_order()
    logging.info(f'Generated {order}')
    kinesis_entry = {
        'Data': json.dumps(order).encode('utf-8'),
        'PartitionKey': order['order_id']
    }
    order_records.append((order, kinesis_entry))
    if len(order_records) > 20:
        try:
            # break apart orders from records as separate lists
            orders, records = map(list, zip(*order_records))

            # write records to Kinesis via PutRecords API
            response = kinesis.put_records(Records=records,
                                           StreamName=stream_name)

            # inspect records to check for any that failed to be written
            to Kinesis
            for i, record_response in enumerate(response['Records']):
                error_code = record_response.get('ErrorCode')
                o = orders[i]
                if error_code:
                    err_msg = record_response['ErrorMessage']
                    logging.error(f"Failed to produce {o} because
{err_msg}")
                else:
                    seq = record_response['SequenceNumber']
                    shard = record_response['ShardId']
                    logging.info(f"Produced {o} sequence {seq} to Shard
{shard}")

            order_records.clear()
```

The steps and code is same as send one record but only different in main.py only.

This Code:

1. Create an Empty list called order_records.
2. Generate an Fake order and create an kinesis entry convert python object into json and encode the order and give partion key and append order and kinesis entry into order_records.
3. Using map split the order_records and order,records.
4. Send records into kinesis using put_records function.
5. Then it checks the response for each record and logs whether it was successfully sent or failed.

Structure of Response:

```
{
  "FailedRecordCount": 0,
  "Records": [
    {
      "SequenceNumber":
"49543463076548007577105092703039560359975228518395019266",
      "ShardId": "shardId-000000000000"
    },
    {
      "ErrorCode": "ProvisionedThroughputExceededException",
      "ErrorMessage": "Rate exceeded for shard shardId-0000000000001 in
stream exampleStreamName under account 123456789012."
    }
    ...
  ],
  "ResponseMetadata": {
    "RequestId": "12345678-1234-1234-1234-123456789012",
    "HTTPStatusCode": 200,
    "HTTPHeaders": {...},
    "RetryAttempts": 0
  }
}
```

6.5 Java: Producing Records with Java and the Kinesis Producer Library

The **Kinesis Producer Library (KPL)** is a high-performance library provided by **Amazon Web Services (AWS)** to help developers **efficiently and reliably send data to Amazon Kinesis Data Streams**.

KPL:

KPL is a client-side library that simplifies and optimizes the process of **publishing data records** into a Kinesis stream. It handles many of the complex tasks involved in data ingestion, such as:

- **Batching:** Groups multiple records together to reduce the number of API calls.
- **Aggregation:** Combines multiple user records into a single Kinesis record to improve throughput.

- **Retries:** Automatically retries failed records.
- **Error handling:** Provides detailed error messages and logs.
- **Asynchronous operation:** Sends data in the background without blocking your application.

This Java code defines a **Kinesis Producer application** using the **Amazon Kinesis Producer Library (KPL)**. It continuously generates fake order data and sends it to a specified Kinesis stream.

Package and Imports

```
package com.thecodinginterface.kinesis;
```

- Declares the package name.

```
import com.amazonaws.services.kinesis.producer.;import
com.fasterxml.jackson.core.JsonProcessingException;import
com.fasterxml.jackson.databind.ObjectMapper;import com.google.common.util.concurrent.;import
org.apache.log4j.*;
```

- Imports necessary classes:
 - **KinesisProducer:** Sends data to Kinesis.
 - **ObjectMapper:** Serializes Java objects to JSON.
 - **Futures & Callbacks:** Handle asynchronous results.
 - **Logger:** Logs messages.

Class and Logger Setup

```
public class ProducerApp { static final Logger logger =
LogManager.getLogger(ProducerApp.class); static final ObjectMapper objectMapper = new
ObjectMapper();
```

- Defines the main class.
- Initializes a logger for logging messages.
- Creates a reusable `ObjectMapper` for JSON serialization.

Main Method

```
public static void main(String[] args) {
```

- Entry point of the application.

```
String streamName = args[0];String region = args[1];
```

- Takes **stream name** and **AWS region** from command-line arguments.

```
logger.info(String.format("Starting Kinesis Producer Library Application for Stream %s in %s",  
streamName, region));
```

- Logs the startup message.

Kinesis Producer Configuration

```
var producerConfig = new KinesisProducerConfiguration().setRegion(region);
```

- Configures the Kinesis producer with the specified AWS region.

```
if (args.length == 3 && args[2].equalsIgnoreCase("--no-agg")) {
```

```
    logger.info("Disabling aggregation");
```

```
    producerConfig.setAggregationEnabled(false);
```

```
}
```

- If the third argument is `--no-agg`, it disables **aggregation** (useful when non-KPL consumers need to read the data).

Producer Initialization

```
var producer = new KinesisProducer(producerConfig);
```

- Creates a new Kinesis producer instance.

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
```

```
    logger.info("Shutting down program");
```

```
    producer.flush();
```

```
}, "producer-shutdown"));
```

- Adds a shutdown hook to **flush** any remaining records before the app exits.

Sending Records in a Loop

```
List<Future<UserRecordResult> putFutures = new LinkedList<>
```

```
());int i = 0;
```

```
while(true) {
```

- Initializes a list to store futures (asynchronous results).
- Starts an infinite loop to continuously send data.

- **List<Future<UserRecordResult>>:**

This declares a list that will hold `Future` objects. Each `Future` represents the result of an **asynchronous operation** — in this case, the result of sending a record to **Amazon Kinesis** using the **Kinesis Producer Library (KPL)**.

- **Future<UserRecordResult>:**

A `Future` is a Java object that acts as a placeholder for a result that will be available in the future.

- `UserRecordResult` is the type of result expected once the asynchronous operation (sending a record to Kinesis) completes.
- So, each `Future<UserRecordResult>` will eventually contain the result of a single record being sent to the Kinesis stream.

- **new LinkedList<>():**

This initializes the list using a `LinkedList` implementation. A `LinkedList` is chosen here likely because:

- It allows fast insertions and deletions.
- The order of insertion is preserved.

```
var order = OrderGenerator.makeOrder();
```

```
logger.info(String.format("Generated %s", order));
```

- Generates a fake order using a helper class and logs it.

```
ByteBuffer data = null;
```

```
try {
```

```
data = ByteBuffer.wrap(objMapper.writeValueAsBytes(order));
```

```
}
```

```
catch (JsonProcessingException e) {
```

```
    logger.error(String.format("Failed to serialize %s", order), e);
```

```
    continue;
```

```
}
```

- Serializes the order to JSON and wraps it in a `ByteBuffer`.
- If serialization fails, logs the error and skips to the next iteration.

```
ListenableFuture<UserRecordResult> future =  
producer.addUserRecord( streamName, order.getOrderID(), data);  
  
putFutures.add(future);
```

- Sends the record to Kinesis using the order ID as the partition key.
- Adds the future result to the list.

```
producer.addUserRecord(...)
```

- This method sends a record to the specified **Kinesis stream**.

```
ListenableFuture<UserRecordResult> future
```

- The `addUserRecord` method returns a `ListenableFuture`, which represents the **pending result** of the asynchronous operation.
- Once the record is successfully sent (or fails), this future will be completed with a `UserRecordResult` or an error.

Handling Async Results

```
Futures.addCallback(future, new FutureCallback<UserRecordResult>() {  
  
    @Override    public void onFailure(Throwable t) {  
  
        logger.error("Failed to produce batch", t);  
  
    }  
  
    @Override    public void onSuccess(UserRecordResult result) {  
  
        logger.info(String.format("Produced User Record to shard %s at position  
%s", result.getShardId(), result.getSequenceNumber()));  
  
    }  
  
},  
  
MoreExecutors.directExecutor());
```

- Registers a callback to handle success or failure of the record send operation.
- Logs the shard ID and sequence number on success.

- `Futures.addCallback(...)`
 - This is a utility method from **Google Guava's Futures class**.
 - It attaches a callback to a `ListenableFuture` (in this case, `future`) so that when the asynchronous operation completes, either the `onSuccess` or `onFailure` method is called.
- `future`
 - This is the `ListenableFuture<UserRecordResult>` returned by `producer.addUserRecord(...)`.
 - It represents the pending result of sending a record to Kinesis.
- `new FutureCallback<UserRecordResult>() { ... }`
 - This is an **anonymous class** implementing the `FutureCallback` interface.
 - It defines two methods:
 - `onSuccess(UserRecordResult result)`: Called when the record is successfully sent to Kinesis.
 - Logs the **shard ID** and **sequence number** where the record was stored.
 - `onFailure(Throwable t)`: Called if the record fails to send.
 - Logs the error with the exception details.
- `MoreExecutors.directExecutor()`
 - This tells Guava to run the callback **in the same thread** that completes the future.
 - It avoids creating a new thread, which is efficient for lightweight callbacks.

Artificial Delay

```
if (++i % 100 == 0) {

    try {        Thread.sleep(500);

    }

    catch (InterruptedException e) {

        logger.warn("Sleep interrupted", e);    }

}
```

- After every 100 records, pauses for 500ms to slow down the output for readability

Tuning Batching Behavior

You can control batching with these settings in `KinesisProducerConfiguration`

```
producerConfig.setAggregationEnabled(true); // default is true
producerConfig.setRecordMaxBufferedTime(100); // in milliseconds
producerConfig.setMaxConnections(1); // number of HTTP connections to Kinesis
producerConfig.setRequestTimeout(60000); // in milliseconds.
```

7. Consume or read data from kinesis data stream

For consuming Records from KDS we use `get_records` function and we need to `shardIterator`

And need to give `shardIteratorType`.

◆ `get-shard-iterator`

Gets a shard iterator for reading data.

```
aws kinesis get-shard-iterator \
  --stream-name <stream-name> \
  --shard-id <shard-id> \
  --shard-iterator-type <type> \
  [--starting-sequence-number <sequence-number>]
```

Shard Iterator Types: `AT_SEQUENCE_NUMBER`, `AFTER_SEQUENCE_NUMBER`, `TRIM_HORIZON`, `LATEST`, `AT_TIMESTAMP`

```
{
  "ShardIterator": "AAAAAAAAAAAAEXAMPLE"
}
```

◆ `get-records`

Gets data records from a shard.

```
aws kinesis get-records \
  --shard-iterator <iterator-token> \
  [--limit <number-of-records>]
{
  "Records": [
    {
      "SequenceNumber":
"49640610668614886128711708916060667677544601327287074818",
      "Data": "dGVzdDE=",
      "PartitionKey": "key1"
    },
    {
      "SequenceNumber":
"49640610668614886128711708916060667677544601327287074819",
      "Data": "dGVzdDI=",
      "PartitionKey": "key2"
    }
  ],
  "NextShardIterator": "AAAAAAAAAAAAEXAMPLENEXT"}
```

7.1 Java : Consuming Records from KDS

Main Consumer Code:

```
package com.thecodinginterface.kinesis;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.log4j.LogManager;
import org.apache.log4j.Logger;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.*;

import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

public class GetRecordsApp {
    static final Logger logger = LogManager.getLogger(GetRecordsApp.class);
    static final ObjectMapper objMapper = new ObjectMapper();

    public static void main(String[] args) {
        logger.info("Starting GetRecords Consumer");
        String streamName = args[0];

        // Instantiate the client
        var client = KinesisClient.builder().build();

        // Add shutdown hook to cleanly close the client when program
terminates
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            logger.info("Shutting down program");
            client.close();
        }, "consumer-shutdown"));

        // Collect list of Shard Id and Shard Iterator Pairs for Fetching
Records Per Shard
        logger.info("Fetching Shards and Iterators");
        var describeStreamReq =
DescribeStreamRequest.builder().streamName(streamName).build();

        List<ShardIteratorPair> shardIterators = new ArrayList<>();
        DescribeStreamResponse describeStreamRes;

        do {
            describeStreamRes = client.describeStream(describeStreamReq);
            for (Shard shard : describeStreamRes.streamDescription().shards())
            {
                // start from last untrimmed record in the shard in the system
```

```

        // (the oldest data record in the shard)
        var itrRequest = GetShardIteratorRequest.builder()
            .streamName(streamName)
            .shardIteratorType(ShardIteratorType.TRIM_HORIZON)
            .shardId(shard.shardId())
            .build();
        var shardItrRes = client.getShardIterator(itrRequest);
        var shardItr = shardItrRes.shardIterator();

        shardIterators.add(new ShardIteratorPair(shard.shardId(),
shardItr));
    }
} while(describeStreamRes.streamDescription().hasMoreShards());

// Continuously Poll the Shards, collecting Records and updating to
use the latest Shard Iterators
while (true) {
    for (var shardItrPair : shardIterators) {

        // construct request to GetRecords up to a limit using the
given Shard Iterator
        var recordsReq = GetRecordsRequest.builder()
            .shardIterator(shardItrPair.getShardItr())
            .limit(200)
            .build();

        // execute GetRecords API call
        GetRecordsResponse recordsRes = client.getRecords(recordsReq);
        shardItrPair.setShardItr(recordsRes.nextShardIterator());

        if (recordsRes.hasRecords()) {
            // iterate over the fetched records, parsing and
deserializing to Order objects
            for (var record : recordsRes.records()) {
                var orderStr = new String(record.data().asByteArray(),
StandardCharsets.UTF_8);
                try {
                    var order = objMapper.readValue(orderStr,
Order.class);
                    logger.info(String.format("Read Order %s from
shard %s at position %s",
                        order,
                        shardItrPair.getShardId(),
                        record.sequenceNumber()));
                } catch (JsonProcessingException e) {
                    logger.error("Failed to deserialize record", e);
                }
            }
        }
    }
}

```



```

    }
    }
}

// introduce artificial pause to allow for log viewing during demo
try {
    Thread.sleep(200);
} catch (InterruptedException e) {
    logger.error("Sleep Interrupted", e);
}
}
}
}

```

Order Class and OrderItem Class is important to print.

Code Breakdown:

var `describeStreamReq` = DescribeStreamRequest.builder().streamName(streamName).build();

- **Creates a request** to describe a Kinesis stream.
- `streamName` is the name of the stream you're querying.
- `DescribeStreamRequest.builder()` is part of the AWS SDK's builder pattern to construct the request.

It Contains this information

```

{
  "StreamDescription": {
    "StreamName": "my-kinesis-stream",
    "StreamARN": "arn:aws:kinesis:us-east-1:123456789012:stream/my-kinesis-stream",
    "StreamStatus": "ACTIVE",
    "Shards": [
      {
        "ShardId": "shardId-000000000000",
        "HashKeyRange": {
          "StartingHashKey": "0",
          "EndingHashKey": "170141183460469231731687303715884105727"
        },
        "SequenceNumberRange": {
          "StartingSequenceNumber":
"49590338271490256608559692538361571095921575989136588898"
        }
      },
      {
        "ShardId": "shardId-000000000001",
        "HashKeyRange": {
          "StartingHashKey": "170141183460469231731687303715884105728",
          "EndingHashKey": "340282366920938463463374607431768211455"
        },
        "SequenceNumberRange": {
          "StartingSequenceNumber":
"49590338271490256608559692538361571095921575989136588899"
        }
      }
    ]
  }
}

```

```

        }
    }
    ],
    "HasMoreShards": false
}
}
},

```

```
List<ShardIteratorPair> shardIterators = new ArrayList<>();
```

- Initializes a **list to store shard iterators**.
- `ShardIteratorPair` is likely a custom class (not part of AWS SDK) that holds a shard ID and its corresponding iterator.
- We need to create a custom class to hold `shardId` and `ShardItr`.

```

package com.thecodinginterface.kinesis;

public class ShardIteratorPair {
    String shardId;
    String shardItr;

    public ShardIteratorPair(String shardId, String shardItr) {
        this.shardId = shardId;
        this.shardItr = shardItr;
    }

    public String getShardId() {
        return shardId;
    }

    public void setShardId(String shardId) {
        this.shardId = shardId;
    }

    public String getShardItr() {
        return shardItr;
    }

    public void setShardItr(String shardItr) {
        this.shardItr = shardItr;
    }
}

```

DescribeStreamResponse describeStreamRes;

- Declares a variable to hold the **response** from the `describeStream` API call.
 - `DescribeStreamResponse` is a **class** provided by the AWS SDK for Java.
-

```
do { describeStreamRes = client.describeStream(describeStreamReq);
```

- Starts a `do-while` loop to **fetch all shards** in the stream.
 - `client.describeStream(...)` sends the request to AWS and gets the stream's metadata, including its shards.
-

```
for (Shard shard : describeStreamRes.streamDescription().shards()) {
```

- Iterates over each **shard** in the stream.
 - A **shard** is a unit of capacity in Kinesis that holds a sequence of data records.
-

```
var itrRequest  
=GetShardIteratorRequest.builder().streamName(streamName).shardIteratorType(ShardIteratorType.TRIM_HORIZON).shardId(shard.shardId()).build();``
```

- Builds a **GetShardIteratorRequest** for each shard.
 - `TRIM_HORIZON` means the iterator will start from the **oldest available record** in the shard.
-

```
var shardItrRes = client.getShardIterator(itrRequest);
```

- Sends the request to get the **shard iterator** from AWS.
 - Which returns a **shard iterator string** — a pointer to the position in the shard from where you can start reading records.
-

```
var shardItr = shardItrRes.shardIterator();
```

- Extracts the **shard iterator string**, which is used to read data from the shard.
-

```
shardIterators.add(new ShardIteratorPair(shard.shardId(), shardItr));
```

- Adds the shard ID and its iterator to the list for later use.
- Create the instance class for `ShardIteratorPair`.

```
} while(describeStreamRes.streamDescription().hasMoreShards());
```

- Continues the loop if there are **more shards** to be fetched (pagination).
- `hasMoreShards()` is true if not all shards were returned in the current response.

```
while (true) {
```

- This creates an **infinite loop** that continuously polls the Kinesis stream for new records.
- It will keep running unless you break it manually or add a condition to exit.

```
for (var shardItrPair : shardIterators) {
```

- Iterates over each `ShardIteratorPair` in the `shardIterators` list.
- Each `shardItrPair` holds:
 - `shardId`: ID of the shard (e.g., "shardId-000000000000")
 - `shardItr`: the current **shard iterator string** used to fetch records from that shard.

```
var recordsReq =
```

```
GetRecordsRequest.builder() .shardIterator(shardItrPair.getShardItr()).limit(200).build();
```

- Creates a request to fetch up to **200 records** from the shard.
- Uses the **current shard iterator** from `shardItrPair`.
- `recordsReq` holds a request like:

```
{
```

```
"ShardIterator": "AAAAAA...EXAMPLE",
```

```
"Limit": 200
```

```
}
```

```
GetRecordsResponse recordsRes = client.getRecords(recordsReq);
```

- Sends the request to AWS Kinesis.
- `recordsRes` now holds the response, which includes:
 - A list of **records** (if any)

- A **next shard iterator** to continue reading from the next position

The `recordsRes` Holds the this values

```
{
  "Records": [
    {
      "Data": "eyJvcnRlcklkIjogMTIzLCAiaXRlbSI6ICJMYXB0b3AifQ==",
      "PartitionKey": "order-1",
      "SequenceNumber":
"49590338271490256608559692538361571095921575989136588898",
      "ApproximateArrivalTimestamp": "2025-10-21T14:00:00Z"
    }
  ],
  "NextShardIterator": "AAAAAAAAAAEXAMPLEuLJ0z1n2vW9v0v9...",
  "MillisBehindLatest": 0
}
```

The `Data` field is base64-encoded. When decoded, it might look like:

```
{"orderId": 123, "item": "Laptop"}
```

```
shardIterPair.setShardIter(recordsRes.nextShardIterator());
```

- Updates the `shardIter` in the pair with the **next iterator**.
 - This ensures the next loop iteration continues reading from where it left off.
-

```
if (recordsRes.hasRecords()) {
```

- Checks if any records were returned in the response.
-

```
for (var record : recordsRes.records()) {
```

- Iterates over each record in the response.
-

```
var orderStr = new String(record.data().asByteArray(), StandardCharsets.UTF_8);
```

- Converts the binary data in the record to a **UTF-8 string**.
 - Example: `{"orderId":123,"item":"Laptop"}`
-

```
var order = objectMapper.readValue(orderStr, Order.class);
```

- Uses Jackson's `ObjectMapper` to convert the JSON string into an `Order` object.

- Example: `Order{id=123, item="Laptop"}`

```
logger.info(String.format("Read Order %s from shard %s at position  
%s",    order,    shardIterPair.getShardId(),    record.sequenceNumber()));
```

- Logs the order details, shard ID, and sequence number.
- Example log: Read Order `Order{id=123, item="Laptop"}` from shard `shardId-000000000000` at position `49590338271490256608559692538361571095921575989136588898`

```
} catch (JsonProcessingException e) {    logger.error("Failed to deserialize record", e);}
```

- If the JSON is malformed or doesn't match the `Order` class, this block logs the error.
-

7.2 Python: Consuming Records from KDS

```
import json
import logging
import sys
import time

import boto3

logging.basicConfig(
    format='%(asctime)s %(name)-12s %(levelname)-8s %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    level=logging.INFO,
    handlers=[
        logging.FileHandler("consumer.log"),
        logging.StreamHandler(sys.stdout)
    ]
)

class ShardIteratorPair:
    def __init__(self, shard_id, iterator):
        self.shard_id = shard_id
        self.iterator = iterator

def main(args):
    logging.info('Starting GetRecords Consumer')
```

```

stream_name = args[1]

# create client
kinesis = boto3.client('kinesis')

# fetch all shards and associated initial iterators
logging.info("Fetching Shards and Iterators")
shard_iterators = []
shards_response = kinesis.list_shards(StreamName=stream_name)
has_more = bool(shards_response['Shards'])
while has_more:
    for shard in shards_response['Shards']:
        shard_id = shard['ShardId']
        # start from last untrimmed record in the shard in the system
        # (the oldest data record in the shard)
        itr_response = kinesis.get_shard_iterator(StreamName=stream_name,
                                                    ShardId=shard_id,
                                                    ShardIteratorType='TRIM_
HORIZON')

        shard_itr = ShardIteratorPair(shard_id,
itr_response['ShardIterator'])
        shard_iterators.append(shard_itr)

    if 'NextToken' in shards_response:
        shards_response = kinesis.list_shards(StreamName=stream_name,
                                                    NextToken=shards_response['N
extToken'])
        has_more = bool(shards_response['Shards'])
    else:
        has_more = False

# continuously Poll shards, collecting records and updating to use latest
shard iterators
while True:
    for shard_itr in shard_iterators:
        try:
            records_response =
kinesis.get_records(ShardIterator=shard_itr.iterator,
                                                                Limit=200)

            for record in records_response['Records']:
                order = json.loads(record['Data'].decode('utf-8'))
                logging.info(f"Read Order {order} from Shard
{shard_itr.shard_id} at position {record['SequenceNumber']}")

            if records_response['NextShardIterator']:
                shard_itr.iterator = records_response['NextShardIterator']

```

```

        except Exception as e:
            logging.error({'message': 'Failed fetching records', 'error':
str(e)})

            time.sleep(0.5)

if __name__ == '__main__':
    main(sys.argv)

```

Steps:

1.Create the ShardIteratorPair class which has shardId and shardItr.

2.shard_response Which holds

```

{
  "Shards": [
    {
      "ShardId": "shardId-000000000000",
      "HashKeyRange": {
        "StartingHashKey": "0",
        "EndingHashKey": "170141183460469231731687303715884105727"
      },
      "SequenceNumberRange": {
        "StartingSequenceNumber":
"49590338271490256608559692538361571095921575989136588898"
      }
    }
  ],
  "NextToken": "eyJTdHJlYWl0YWl1Ijoi..."
}

```

3.has_more variable hold true or false depends it has shards or not.

4.get_shard_iterator function used to get iterator.

5.Loop through the records

6.record_response which holds

```

{
  "Records": [
    {
      "Data": "eyJvcnRlcklkIjogMTIzLCAiaXRlbSI6ICJMYXB0b3AifQ==",
      "PartitionKey": "order-1",
      "SequenceNumber":
"49590338271490256608559692538361571095921575989136588898",
      "ApproximateArrivalTimestamp": "2025-10-21T14:00:00Z"
    }
  ],
  "NextShardIterator": "AAAAAAAAAEXAMPLEuLJ0z1n2vW9v0v9...",
  "MillisBehindLatest": 0
}

```

7.update the shard_itr.iterator with NextShardIterator

8.Decode the data and convert into python object.

7.3 Problems of GetRecords based SDK Consumers

1. Consumed records need to be tracked in some external data store to avoid duplicate processing
2. Multiple instance of the same application became hard.
3. Shard Splitting and Shard Merging requires Complex logic.
4. Cannot handle de-aggregation of KPL produced records.
5. KCL(kinesis Client Library) , AWS Lambda Consumer, Kinesis Analytics(SQL , Apache Flink) is best solution.

Scenario: Real-Time Log Processing System

Imagine you're building a **log analytics system** for a large-scale web application. Logs are continuously pushed into an **Amazon Kinesis Data Stream**. You want to process these logs in real-time using multiple Java applications running on different EC2 instances.

◆ 1. Load Balancing Across Multiple Consumer Instances

Problem: You have multiple shards in your Kinesis stream and multiple consumer applications (workers). How do you ensure each worker gets a fair share of the workload?

KCL Solution: KCL uses a **worker coordination mechanism** (via DynamoDB) to **automatically assign shards** to different workers. If you launch 3 EC2 instances running the same KCL app, KCL will distribute the shards among them.

Example:

- Stream has 6 shards.
- You run 3 KCL workers.
- KCL assigns 2 shards to each worker.

If a worker crashes, KCL rebalances the shards among the remaining workers.

◆ 2. Checkpointing to Track Progress

Problem: How do you ensure that your application doesn't reprocess the same data or miss data if it restarts?

KCL Solution: KCL uses **DynamoDB** to store **checkpoints**—the last successfully processed record for each shard. This allows the app to resume from where it left off.

Example:

- Worker processes records up to sequence number 12345.
- It checkpoints this value.
- If the worker restarts, it reads from 12346 onward.

◆ 3. Error Handling and Retries

Problem: What if processing a record fails due to a transient error (e.g., network issue)?

KCL Solution: KCL allows you to implement **custom retry logic** in your record processor. You can catch exceptions, log them, and retry processing.

Example:

```
public void processRecords(List<Record> records, IRecordProcessorCheckpoint checkpoint) {
    for (Record record : records) {
        try {
            process(record); // your logic
        } catch (Exception e) {
            log.error("Error processing record", e);
            // Optionally retry or skip
        }
    }
    checkpoint.checkpoint(); // Save progress
}
```

◆ 4. Shard Management (Resharding Events)

Problem: What happens if the stream is resharded (e.g., split or merged shards)?

KCL Solution: KCL **automatically detects resharding** and updates the shard assignments. You don't need to manually handle shard changes.

Example:

- You split a shard into two.
- KCL detects the new shards and assigns them to workers.
- The old shard is marked as closed.

♦ 5. Multithreading for Parallel Processing of Shards

Problem: How do you process multiple shards in parallel?

KCL Solution: Each shard is processed in a **separate thread**. KCL handles thread management internally.

Example:

- Worker assigned 3 shards.
 - KCL spawns 3 threads, each running a `RecordProcessor` for one shard.
-

7.4 Java : Consuming Records with Java and the Kinesis Client Library

 KCL Architecture Overview

Here's how a typical KCL-based application works:

1. **Kinesis Data Stream:** Holds the incoming data (e.g., logs, events, metrics).
2. **KCL Worker:** A Java application instance that reads data from one or more shards.
3. **DynamoDB:** Used by KCL to store checkpoints and manage shard leases.
4. **CloudWatch:** Used for monitoring metrics like throughput and errors.

 How KCL Works Internally

1. **Initialization:** Each worker registers itself and discovers the stream's shards.
2. **Shard Assignment:** KCL assigns shards to workers using a lease system stored in DynamoDB.
3. **Record Processing:** Each shard is processed in a separate thread using a `RecordProcessor`.
4. **Checkpointing:** After processing, the worker checkpoints the last processed record.
5. **Resharding Handling:** If shards are split or merged, KCL automatically rebalances them.

KCL Consumer Main Code starting Point:

```
package com.thecodinginterface.kinesis;

import org.apache.log4j.LogManager;
```

```

import org.apache.log4j.Logger;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchAsyncClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
import software.amazon.awssdk.services.kinesis.KinesisAsyncClient;
import software.amazon.kinesis.common.ConfigsBuilder;
import software.amazon.kinesis.common.KinesisClientUtil;
import software.amazon.kinesis.coordinator.Scheduler;

import java.util.UUID;

public class KclConsumerApp {
    static final Logger logger = LogManager.getLogger(KclConsumerApp.class);

    public static void main(String[] args) {

        var schedulerId = UUID.randomUUID().toString();
        String appName = args[0]; // group of the same type of application
        String streamName = args[1];
        Region region = Region.of(args[2]);

        logger.info(String.format("Starting KCL Scheduler %s app %s for stream %s in region %s", schedulerId, appName, streamName, region.id()));

        var kinesisClient =
KinesisClientUtil.createKinesisAsyncClient(KinesisAsyncClient.builder().region(
region));
        var dynamoClient =
DynamoDbAsyncClient.builder().region(region).build();
        var cwClient = CloudWatchAsyncClient.builder().region(region).build();
        var orderProcessorFactory = new OrderRecordProcessorFactory();

        var configsBuilder = new ConfigsBuilder(
            streamName,
            appName,
            kinesisClient,
            dynamoClient,
            cwClient,
            schedulerId,
            orderProcessorFactory
        );

        var scheduler = new Scheduler(
            configsBuilder.checkpointConfig(),
            configsBuilder.coordinatorConfig(),
            configsBuilder.leaseManagementConfig(),
            configsBuilder.lifecycleConfig(),
            configsBuilder.metricsConfig(),

```

```

        configsBuilder.processorConfig(),
        configsBuilder.retrievalConfig()
    );

    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
        logger.info("Shutting down consumer application");
        scheduler.shutdown();
    }, "consumer-shutdown"));

    int exitCode = 0;
    try {
        scheduler.run();
    } catch (Exception e) {
        logger.error(String.format("KCL Scheduler %s app %s encountered
error.", schedulerId, appName), e);
        exitCode = 1;
    }
    System.exit(exitCode);
}
}

```

◆ KclConsumerApp.java

This is the **main application class** that sets up and runs the KCL Scheduler.

✓ Purpose:

To **initialize and start** the KCL Scheduler which:

- Connects to Kinesis
- Assigns shards to processors
- Starts processing records

🔍 Key Components:

String appName = args[0];

String streamName = args[1];

Region region = Region.of(args[2]);

- Takes command-line arguments for:
 - Application name (used for tracking leases in DynamoDB)
 - Kinesis stream name
 - AWS region

var kinesisClient = KinesisClientUtil.createKinesisAsyncClient(...);

```
var dynamoClient = DynamoDbAsyncClient.builder().region(region).build();
var cwClient = CloudWatchAsyncClient.builder().region(region).build();
``
```

- Creates async clients for:
 - Kinesis (to read data)
 - DynamoDB (for checkpointing and lease management)
 - CloudWatch (for metrics)

```
var configsBuilder = new ConfigsBuilder(...);
```

- Builds all the configuration objects needed by the KCL Scheduler.

```
var scheduler = new Scheduler(...);
```

- Creates the **Scheduler**, which is the core component that:
 - Manages shard leases
 - Assigns processors
 - Handles retries, checkpointing, and lifecycle events

```
scheduler.run();
```

```
``
```

- Starts the scheduler loop to begin processing records.

```
Runtime.getRuntime().addShutdownHook(...);
```

- Ensures the scheduler shuts down gracefully when the app is stopped.

KclConsumerApp starts the application and sets up the KCL Scheduler.

OrderRecordProcessorFactory Class:

```
package com.thecodinginterface.kinesis;

import software.amazon.kinesis.processor.ShardRecordProcessor;
import software.amazon.kinesis.processor.ShardRecordProcessorFactory;

/**
 * Simple factor for providing instance of ShardRecordProcessor to the
 * Scheduler
 * to lease to shards.
 */
public class OrderRecordProcessorFactory implements
ShardRecordProcessorFactory {
```

```

@Override
public ShardRecordProcessor shardRecordProcessor() {
    return new OrderRecordProcessor();
}
}

```

◆ OrderRecordProcessorFactory.java

public class OrderRecordProcessorFactory implements ShardRecordProcessorFactory {

 @Override

 public ShardRecordProcessor shardRecordProcessor() {

 return new OrderRecordProcessor();

 }

}

✅ Purpose:

This is a **factory class** that creates instances of `OrderRecordProcessor`.

🔍 Why it's needed:

- The **KCL Scheduler** needs a way to create a new `ShardRecordProcessor` for each shard it processes.
- This factory provides that mechanism.

The Scheduler uses `OrderRecordProcessorFactory` to create a new `OrderRecordProcessor` for each shard.

OrdersManager Class:

```

package com.thecodinginterface.kinesis;

import java.util.*;

public class OrdersManager {

    private final Set<Order> orders;

    public OrdersManager() {
        this.orders = new HashSet<>();
    }
}

```

```

public void addOrder(Order order) {
    orders.add(order);
}

public double calcAverageOrderSize() {
    double avgOrderSize = 0.0;
    if (!orders.isEmpty()) {
        double totalSales = orders.stream()
            .flatMap(o -> o.getOrderItems().stream())
            .map(orderItem -> orderItem.getProductPrice() *
orderItem.getProductQuantity())
            .reduce(0.0, Double::sum);
        avgOrderSize = totalSales / orders.size();
    }
    return avgOrderSize;
}
}

```

◆ OrdersManager.java

✓ Purpose:

This class **aggregates order data** and calculates the **average order size**.

🔍 Key Functions:

- `addOrder(Order order)`: Adds an order to the internal set.
- `calcAverageOrderSize()`:
 - Loops through all orders.
 - Calculates total sales by multiplying price × quantity for each item.
 - Divides total sales by the number of orders to get the average.

Each `OrderRecordProcessor` uses **OrdersManager** to:

- Collect orders
- Calculate average order size
- Log and checkpoint progress

OrderRecordProcessor Class (Initialization):

📦 Class Overview

```
package com.thecodinginterface.kinesis;
```


- Declares the package name for organizing the class.

Imports

```
import com.fasterxml.jackson.databind.ObjectMapper;  
``
```

- Used for converting JSON data into Java objects (deserialization).

```
import org.apache.log4j.LogManager;  
import org.apache.log4j.Logger;  
``
```

- For logging messages (info, error, etc.).

```
import software.amazon.kinesis.exceptions;  
import software.amazon.kinesis.lifecycle.events;  
import software.amazon.kinesis.processor.*;  
import software.amazon.kinesis.retrieval.KinesisClientRecord;
```

- These are KCL classes for handling lifecycle events, exceptions, and record processing.

Class Definition

```
public class OrderRecordProcessor implements ShardRecordProcessor {
```

- Implements the `ShardRecordProcessor` interface, which defines how to process records from a single shard.

```
static final Logger logger = LogManager.getLogger(OrderRecordProcessor.class);
```

- Logger for printing logs.

```
private String shardId;
```

```
private OrdersManager ordersMgr = new OrdersManager();
```

- `shardId`: ID of the shard being processed.
- `ordersMgr`: Helper class to manage and aggregate order data.
- Create the instance of `orderManager` class.

```
static final long REPORTING_INTERVALMS = 30000L;
```

```
private long nextReportingTimeMS;
```

- Defines a 30-second reporting interval for calculating average order size.

```
static final long MAXRETRYSLEEPMS = 640000L;
```

```
static final long DEFAULTRETRYSLEEPMS = 10000L;
```

```
private long retrySleepMs = DEFAULTRETRYSLEEP_MS;
```

- Constants for retry logic in case of checkpointing failures.

```
1. MAX_RETRY_SLEEP_MS = 640000L
```

- This sets the **maximum backoff time** (in milliseconds) for retrying a failed checkpoint.
- **640,000 ms = 10 minutes and 40 seconds.**
- It prevents the application from retrying indefinitely with increasing delays.

```
2. DEFAULT_RETRY_SLEEP_MS = 10000L
```

- This is the **initial wait time** (10 seconds) before retrying a failed checkpoint.
- If the checkpoint fails due to throttling or transient issues, the app waits this long before retrying.

```
3. retrySleepMs = DEFAULT_RETRY_SLEEP_MS
```

- This is a **mutable variable** that starts at the default value and **doubles** with each retry (exponential backoff).
- It resets to the default after a successful checkpoint.

```
private final ObjectMapper objMapper = new ObjectMapper();
```

- Jackson object mapper for converting JSON to Java objects.

Lifecycle Methods

```
1. initialize()
```

```
public void initialize(InitializationInput initializationInput) {  
    shardId = initializationInput.shardId();  
    logger.info(String.format("Initialized shard %s @ sequence %s",  
        shardId, initializationInput.extendedSequenceNumber().toString()));  
    nextReportingTimeMS = System.currentTimeMillis() + REPORTING_INTERVALMS;  
}
```

- Called when the processor starts for a shard.
- Logs the shard ID and sequence number.
- Sets the next reporting time.

2. processRecords()

```
public void processRecords(ProcessRecordsInput processRecordsInput) {
```

- Called repeatedly with batches of records from the shard.

```
for (KinesisClientRecord record : processRecordsInput.records()) {
```

```
    byte[] byteArr = new byte[record.data().remaining()];
```

```
    record.data().get(byteArr);
```

- Reads the raw byte data from each record.

```
    try {
```

```
        ordersMgr.addOrder(objMapper.readValue(byteArr, Order.class));
```

```
    } catch (IOException e) {
```

```
        logger.error(String.format("Shard %s failed to deserialize record", shardId), e);
```

```
    }
```

```
}
```

- Deserializes the JSON into an `Order` object and adds it to the `OrdersManager`.

```
if (System.currentTimeMillis() > nextReportingTimeMS) {
```

```
    double avgOrderSize = ordersMgr.calcAverageOrderSize();
```

```
    nextReportingTimeMS = System.currentTimeMillis() + REPORTING_INTERVALMS;
```

```
    ordersMgr = new OrdersManager();
```

```
    logger.info(String.format("Shard %s Average Order %.2f", shardId, avgOrderSize));
```

```
    checkpoint(processRecordsInput.checkpointer());
```

```
}
```

```
}
```

- Every 30 seconds:
 - Calculates the average order size.
 - Logs it.
 - Resets the `OrdersManager`.

- Performs a **checkpoint** to save progress.

3. leaseLost()

```
public void leaseLost(LeaseLostInput leaseLostInput) {  
    logger.info(String.format("Shard %s lease lost", shardId));  
}
```

- Called when the worker loses the lease for the shard.
- Logs the event.

4. shardEnded()

```
public void shardEnded(ShardEndedInput shardEndedInput) {  
    logger.info(String.format("Shard %s end reached, doing final checkpoint", shardId));  
    checkpoint(shardEndedInput.checkpointer());  
}
```

- Called when the shard is closed (e.g., due to resharding).
- Final checkpoint is required to allow processing of child shards.

5. shutdownRequested()

```
public void shutdownRequested(ShutdownRequestedInput shutdownRequestedInput) {  
    logger.info(String.format("Shard %s Scheduler is shutting down, checkpointing", shardId));  
    checkpoint(shutdownRequestedInput.checkpointer());  
}
```

- Called when the worker is shutting down.
- Performs a final checkpoint.

Checkpointing Logic

```
private void checkpoint(RecordProcessorCheckpointer checkpointer) {
```

- Attempts to save the current position in the stream.

```
    try {  
        checkpointer.checkpoint();
```

- Tries to checkpoint.

```
} catch (ShutdownException se) {
```

```
    logger.info("Caught shutdown exception, skipping checkpoint.", se);
```

- If the processor is shutting down, skip checkpointing.

```
} catch (ThrottlingException e) {
```

```
    logger.error("Throttling exception occurred", e);
```

- If throttled, retry with exponential backoff.

```
if (retrySleepMs < MAXRETRYSLEEP_MS) {
```

```
    try {
```

```
        Thread.sleep(retrySleepMs);
```

```
        retrySleepMs = retrySleepMs * 2;
```

```
        checkpoint(checkpointer);
```

```
    } catch (InterruptedException interruptedException) {
```

```
        interruptedException.printStackTrace();
```

```
        logger.warn(String.format("Retry thread interrupted for shard %s and sleep %d",
shardId, retrySleepMs));
```

```
    }
```

```
    } else {
```

```
        logger.error(String.format("Shard %s failed to perform checkpoint after backoff and
retries exhausted", shardId));
```

```
        Runtime.getRuntime().halt(1);
```

```
    }
```

- Retries checkpointing with increasing delay.
- If retries are exhausted, halts the JVM.

```
} catch (InvalidStateException e) {
```

```
    logger.error("Cannot save checkpoint to the DynamoDB table used by the Amazon Kinesis
Client Library.", e);
```

```
}
```

```
retrySleepMs = DEFAULTRETRYSLEEP_MS;
```

```
}
```

- Handles DynamoDB issues.
- Resets retry delay.

Purpose of This Class

This class is a **KCL record processor** that:

- Reads order data from a Kinesis stream.
 - Aggregates it using a helper class.
 - Calculates average order size every 30 seconds.
 - Logs the result.
 - Checkpoints progress to ensure reliable processing.
-

WorkFlow:

High-Level Flow

1. **We run the `KclConsumerApp` with arguments.**
2. It sets up AWS clients and the `KCL Scheduler`.
3. The `Scheduler` uses `OrderRecordProcessorFactory` to create a `OrderRecordProcessor` for each shard.
4. Each `OrderRecordProcessor`:
 - Initializes
 - Processes records
 - Calculates average order size
 - Checkpoints progress
5. The app runs continuously until shutdown.

Step-by-Step Execution

Step 1: Run the Application

we run the app from the command line:

```
java -jar your-kcl-app.jar <appName> <streamName> <region>
```

- ◆ What the arguments mean:

Argument	Example	Description
appName	OrderAnalyticsApp	Logical name for your app (used in DynamoDB for lease tracking)

Argument	Example	Description
streamName	OrderStream	Name of your Kinesis Data Stream
region	ap-south-1	AWS region where your stream and DynamoDB table are located

✅ Step 2: KclConsumerApp Starts

```
var schedulerId = UUID.randomUUID().toString();
```

- Generates a unique ID for this instance of the scheduler.

```
var kinesisClient = KinesisClientUtil.createKinesisAsyncClient(...);
```

- Creates an async client to read from Kinesis.

```
var dynamoClient = DynamoDbAsyncClient.builder().region(region).build();
```

- Used for checkpointing and lease management.

```
var cwClient = CloudWatchAsyncClient.builder().region(region).build();
```

- Sends metrics to CloudWatch.

```
var orderProcessorFactory = new OrderRecordProcessorFactory();
```

- Factory that creates `OrderRecordProcessor` instances.

```
var configsBuilder = new ConfigsBuilder(...);
```

- Builds all the configuration objects needed by the KCL `Scheduler`.

```
var scheduler = new Scheduler(...);
```

- The **core engine** that manages shard assignment, record processing, checkpointing, etc.

```
scheduler.run();
```

- Starts the scheduler loop — this is where the magic begins!

✅ Step 3: Scheduler Assigns Shards

- The `Scheduler` checks the stream and DynamoDB to determine which shards this instance should process.
- For each assigned shard, it calls:

`OrderRecordProcessorFactory.shardRecordProcessor()`

Which returns:

`new OrderRecordProcessor()`

✓ Step 4: `OrderRecordProcessor` Lifecycle

◆ `initialize(InitializationInput input)`

- Called once per shard.
- Sets `shardId` and `nextReportingTimeMS`.

◆ `processRecords(ProcessRecordsInput input)`

- Called repeatedly with batches of records.
- For each record:
 - Deserializes JSON into `Order` object.
 - Adds it to `OrdersManager`.
- Every 30 seconds:
 - Calculates average order size.
 - Logs the result.
 - Checkpoints progress.

◆ `leaseLost(LeaseLostInput input)`

- Called if this worker loses the shard lease.
- Logs the event.

◆ `shardEnded(ShardEndedInput input)`

- Called when the shard is closed (e.g., due to resharding).
- Final checkpoint is required.

◆ `shutdownRequested(ShutdownRequestedInput input)`

- Called when the app is shutting down.
- Performs a final checkpoint.

✓ Step 5: `OrdersManager` in Action

`OrdersManager ordersMgr = new OrdersManager();`

- Holds a `Set<Order>` to avoid duplicates.


```
ordersMgr.addOrder(order);
```

- Adds each order to the set.

```
ordersMgr.calcAverageOrderSize();
```

- Calculates average order size:
 - Loops through all `OrderItems` in each `Order`.
 - Computes total sales = price × quantity.
 - Divides by number of orders.

✓ Step 6: Order and OrderItem Classes

`Order` class:

```
Order {  
    String orderID;  
    String customerID;  
    String sellerID;  
    List<OrderItem> orderItems;  
}
```

`OrderItem` class:

```
OrderItem {  
    String productName;  
    String productCode;  
    int productQuantity;  
    double productPrice;  
}
```

These are deserialized from the JSON records in the stream using Jackson:

```
Order order = objectMapper.readValue(byteArr, Order.class);
```

1. **we run the app** with 3 arguments.
2. **KclConsumerApp** sets up AWS clients and the KCL Scheduler.
3. **Scheduler** starts and assigns shards.
4. For each shard:
 - `OrderRecordProcessorFactory` creates a `OrderRecordProcessor`.
 - `initialize()` is called.
 - `processRecords()` is called repeatedly.
 - Every 30 seconds, average order size is calculated and logged.
 - Checkpointing is done to save progress.
5. On shutdown or shard end, `shutdownRequested()` or `shardEnded()` is called to checkpoint.

 KPL (Producer) → Kinesis Data Stream → KCL (Consumer)

◆ KPL (Kinesis Producer Library)

- A high-performance library for **writing data** to Kinesis Data Streams.
- It batches, compresses, and aggregates multiple user records into a single Kinesis record to improve throughput and reduce costs.

◆ Kinesis Data Stream

- A managed service that stores and transports real-time data.
- Holds the data in **shards** for up to 7 days (default retention).
- Acts as the **bridge** between producers (like KPL) and consumers (like KCL).

◆ KCL (Kinesis Client Library)

- A library for **reading and processing** data from Kinesis Data Streams.
- Automatically handles:
 - Shard assignment
 - Checkpointing
 - Resharding
 - Parallel processing
- **Can read both aggregated and non-aggregated records.**

If we are using KPL to aggregate records, then KCL automatically deaggregates them when consuming.

7.5 Consume Records using AWS Lambda

To get records from **Amazon Kinesis** using **AWS Lambda**, you can use **Kinesis as an event source** for your Lambda function.

- Kinesis streams continuously receive data (e.g., logs, events).
- AWS Lambda can be configured to **poll the Kinesis stream**.
- When new records are available, Lambda **invokes your function** with a batch of records.
- Your function processes the records and returns successfully.

◆ Steps to Configure

1. Create a Kinesis Data Stream

- Go to **AWS Management Console** → **Kinesis** → **Data Streams**.
- Create a new stream (e.g., `MyKinesisStream`).

2. Create a Lambda Function

- Go to **AWS Lambda** → **Create Function**.
- Choose **Author from scratch**.
- Runtime: **Python, Node.js, or Java** (any supported language).
- Example name: `KinesisProcessorLambda`.

3. Add Kinesis as Event Source

- In the Lambda function, go to **Configuration** → **Triggers** → **Add Trigger**.
- Select **Kinesis**.
- Choose your stream (`MyKinesisStream`).
- Set **Batch size** (e.g., 100 records).
- Set **Starting position**:
 - `TRIM_HORIZON` → from oldest record.
 - `LATEST` → from new records only.

4. Write Lambda Code

Example in **Python**:

```
import json
def lambda_handler(event, context):
    for record in event['Records']:
        # Kinesis data is base64 encoded
        payload = record['kinesis']['data']
        decodeddata = json.loads(base64.b64decode(payload).decode('utf-8'))
        print(f"Decoded record: {decoded_data}")
    return {'statusCode': 200}
```

Example in **Java** (using AWS Lambda Java runtime):

```
public class KinesisLambdaHandler implements RequestHandler<KinesisEvent, String> {
    @Override
    public String handleRequest(KinesisEvent event, Context context) {
        for (KinesisEvent.KinesisEventRecord rec : event.getRecords()) {
            String data = new String(rec.getKinesis().getData().array());
            System.out.println("Decoded record: " + data);
        }
        return "Processed " + event.getRecords().size() + " records.";
    }
}
```

5. IAM Role Permissions

- Lambda needs permissions to:
 - Read from Kinesis (`kinesis:GetRecords`, `kinesis:GetShardIterator`, `kinesis:DescribeStream`).
 - Write logs to CloudWatch.

✓ Key Points

- Lambda **polls** the stream automatically (you don't need to manage consumers).
- **Batch size** and **parallelization factor** control throughput.
- Lambda **checkpoints automatically** (you don't need DynamoDB like KCL).
- **Retry behavior**: If processing fails, Lambda retries until success or record expires (24 hours).

If we use **Kinesis Producer Library (KPL)** to produce records, the consumer side (Lambda or KCL) still works the same way, but there are a few important considerations:

🔍 Impact on Lambda Consumer

When Lambda reads from Kinesis:

- Each **Kinesis record** may contain **multiple user records** (because KPL aggregates them).
- Lambda receives the **aggregated record** as a Base64-encoded blob.
- You need to **deaggregate** the record to get the original user records.

🔧 How to Deaggregate in Lambda

AWS provides a **Kinesis Aggregation/Deaggregation library** for this.

Python Example

```
import base64
import json
from awskinesisagg.deagggregator import deagggregaterecords

def lambda_handler(event, context):
    # Deaggregate KPL aggregated records
    userrecords = deagggregaterecords(event['Records'])
    for record in user_records:
        payload = base64.b64decode(record['kinesis']['data']).decode('utf-8')
        data = json.loads(payload)
        print(f"User record: {data}")
```

Java Example

Use the **Amazon Kinesis Aggregators** library:

```
import com.amazonaws.services.kinesis.clientlibrary.types.UserRecord;

for (KinesisEvent.KinesisEventRecord rec : event.getRecords()) {
    for (UserRecord userRecord : UserRecord.deaggregate(rec.getKinesis())) {
        String data = new String(userRecord.getData().array());
        System.out.println("User record: " + data);
    }
}
```

✓ Key Points

- **Without deaggregation**, you will see a single large blob instead of individual records.
 - Lambda **does not automatically deaggregate** KPL records.
 - If you don't want to deal with deaggregation, you can disable aggregation in KPL (but that reduces efficiency).
-

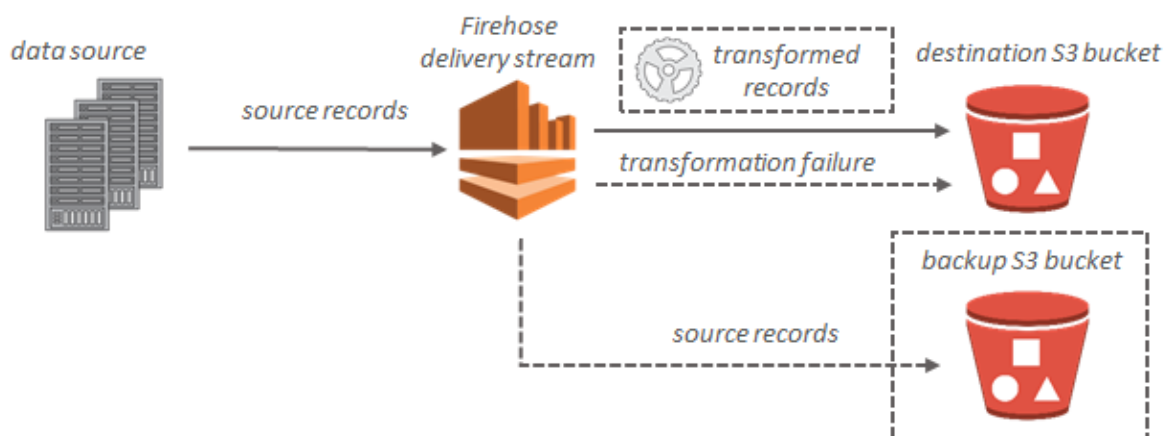
8. Amazon Data Firehose

We Create Delivery Stream For send data from Source to Destination in Kinesis Service or using firehose console.

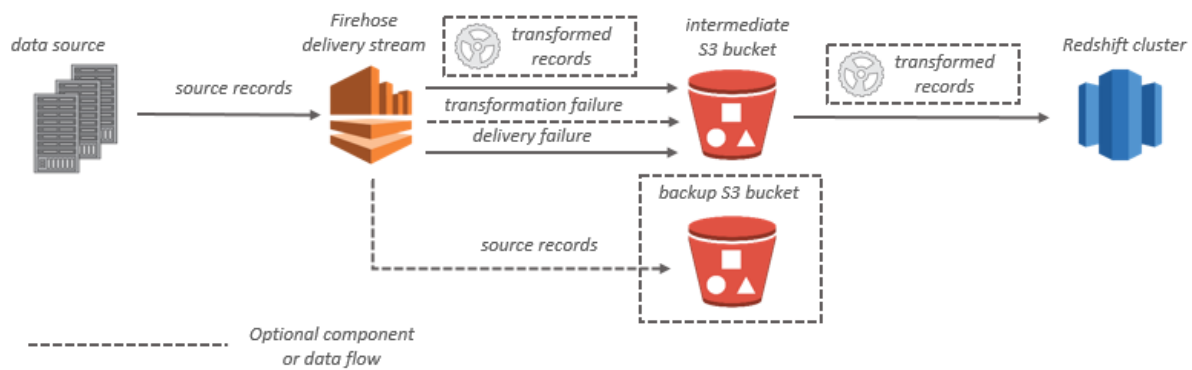
- **Fully managed service** for delivering **real-time streaming data**.
- Supports delivery to multiple destinations:
 - **Amazon S3**
 - **Amazon Redshift**
 - **Amazon OpenSearch Service & Serverless**
 - **Splunk**
 - **Apache Iceberg Tables**
 - **Custom HTTP endpoints**
 - **Third-party services** like Datadog, Dynatrace, LogicMonitor, MongoDB, New Relic, Coralogix, Elastic.
- **No need to write applications or manage infrastructure.**
- You just **configure data producers** to send data to Firehose.
- Firehose **automatically delivers** data to the specified destination.
- You can also **transform data before delivery**.

Data flow in Amazon Data Firehose:

For Amazon S3 destinations, streaming data is delivered to your S3 bucket. If data transformation is enabled, you can optionally back up source data to another Amazon S3 bucket.



For Amazon Redshift destinations, streaming data is delivered to your S3 bucket first. Amazon Data Firehose then issues an Amazon Redshift **COPY** command to load data from your S3 bucket to your Amazon Redshift cluster. If data transformation is enabled, you can optionally back up source data to another Amazon S3 bucket.



8.1 Create a Firehose stream

STEP 1:

Choose source and destination for your Firehose stream

1. Open the Firehose console at <https://console.aws.amazon.com/firehose/>.
2. Choose **Create Firehose stream**.
3. On the **Create Firehose stream** page, choose a source for your Firehose stream from one of the following options.
 - **Direct PUT** – Choose this option to create a Firehose stream that producer applications write to directly.

✔ Use Direct PUT When:

- You have a **simple data ingestion setup**.
- Your application can **directly call Firehose APIs** (`PutRecord`, `PutRecordBatch`).
- **Low to moderate data volume** (up to 5,000 records/sec or 5 MB/sec).
- You don't need **event ordering**.
- You want to **avoid managing infrastructure** like Kinesis Data Streams.
- You need **quick integration** with Firehose from custom apps or scripts.

✘ Do Not Use Direct PUT When:

- You need to handle **high throughput** (above Firehose limits).
- You require **event ordering** or **partitioning**.
- You want **advanced processing** (e.g., windowing, stateful operations).
- You have **multiple distributed producers**—managing direct API calls becomes complex.
- You need **built-in retry, durability, and buffering**—Kinesis Data Streams is better.
- You want to **decouple producers from delivery logic** for scalability.

- AWS SDK
- AWS Lambda
- AWS CloudWatch Logs
- AWS CloudWatch Events
- AWS Cloud Metric Streams
- AWS IoT
- AWS Eventbridge
- Amazon Simple Email Service
- Amazon SNS
- AWS WAF web ACL logs
- Amazon API Gateway - Access logs
- Amazon Pinpoint
- Amazon MSK Broker Logs
- Amazon Route 53 Resolver query logs
- AWS Network Firewall Alerts Logs
- AWS Network Firewall Flow Logs
- Amazon ElastiCache Redis SLOWLOG
- Kinesis Agent (linux)
- Kinesis Tap (windows)
- Fluentbit
- Fluentd
- Apache Nifi
- Snowflake
- **Amazon Kinesis Data Streams** – Choose this option to configure a Firehose stream that uses a Kinesis data stream as a data source. You can then use Firehose to read data easily from an existing Kinesis data stream and load it into destinations.

4. Choose a destination for your Firehose stream from one of the following destinations that Firehose supports.

- Amazon OpenSearch Service
- Amazon OpenSearch Serverless
- Amazon Redshift
- Amazon S3
- Apache Iceberg Tables
- Coralogix
- Datadog
- Dynatrace
- Elastic

- HTTP Endpoint
 - Honeycomb
 - Logic Monitor
 - Logz.io
 - MongoDB Cloud
 - New Relic
 - Splunk
 - Splunk Observability Cloud
 - Sumo Logic
 - Snowflake
5. For **Firehose stream name**, you can either use the name that the console generates for you or add a Firehose stream of your choice.

STEP 2:

Configure source settings for Amazon Kinesis Data Streams:

- **Source Configuration:**
 - You can select an existing Kinesis stream or enter its ARN.
 - If no stream exists, you can create one via the Kinesis console.
- **IAM Permissions:**
 - You need an IAM role with permissions to access the Kinesis stream and S3 destination.
- **Firehose Behavior with Kinesis Source:**
 - **PutRecord** and **PutRecordBatch** operations of Firehose are **disabled**.
 - You must use **Kinesis Data Streams' PutRecord/PutRecords** to send data.
- **Data Reading Position:**
 - Firehose starts reading from the **LATEST** position in the stream.
- **Polling Frequency:**
 - Firehose calls **GetRecords** **once per second per shard**.
 - If **full backup is enabled**, it calls **twice per second per shard**.
- **Multiple Consumers:**
 - Multiple Firehose streams and other consumers can read from the same Kinesis stream.

- Each read counts against the **shard's throttling limit**, so plan usage carefully.

Amazon Data Firehose **does not automatically deaggregate** records that were aggregated by KPL.

KPL aggregates multiple user records into a single Kinesis record for efficiency. Firehose expects **individual records**, so if you send aggregated records directly from KPL to Firehose (via Kinesis Data Streams), you need to **deaggregate them manually**.

We need to **deaggregate the records** before Firehose can process them properly. Here's how:

✅ Option 1: Use the Kinesis Client Library (KCL) or AWS SDK

- Use KCL or AWS SDK to **read from Kinesis Data Streams**.
- Apply **deaggregation logic** using the `RecordDeaggregator` utility (available in AWS SDKs).
- Then **send the deaggregated records to Firehose**.

✅ Option 2: Use AWS Lambda (Recommended)

- Set up a **Lambda function** as a **consumer of Kinesis Data Streams**.
- In the Lambda, use the **KPL deaggregation library** to split records.
- Then **forward the deaggregated records to Firehose**.

AWS Lambda function in Python that:

1. **Consumes records from Kinesis Data Streams.**
2. **Deaggregates KPL records** using the AWS deaggregation library.
3. **Sends the deaggregated records to Amazon Data Firehose.**

✅ Prerequisites:

- Kinesis stream is the source.
- Firehose delivery stream is the destination.
- IAM role with permissions for Kinesis, Firehose, and Lambda.
- AWS SDK (`boto3`) and KPL deaggregation library (`aws_kinesis_agg`) installed.

AWS Lambda Code:

```
import base64
import boto3

from aws_kinesis_agg.deaggregator import deaggregate_records

firehose_client = boto3.client('firehose')

def lambda_handler(event, context):
    # Decode and deaggregate KPL records
    user_records = deaggregate_records(event['Records'])
```

```

for record in user_records:
    payload = base64.b64decode(record['kinesis']['data'])

    # Send to Firehose
    response = firehose_client.put_record(
        DeliveryStreamName='your-firehose-stream-name',
        Record={'Data': payload}
    )

return {
    'statusCode': 200,
    'body': f'Successfully processed {len(user_records)} records.'
}

```

- Replace 'your-firehose-stream-name' with our actual Firehose stream name.
 - Make sure the Lambda function has access to the **Kinesis stream** and **Firehose delivery stream**.
 - We can also batch records using `put_record_batch` for better performance.
-

STEP 3:

Configure record transformation and format conversion:

If you choose Amazon Kinesis Data Streams or Direct PUT as the source for your Firehose stream

In the Source settings section, provide the following fields.

1. Under **Transform records**, choose one of the following:
 - a. If your destination is Amazon S3 or Splunk, in the **Decompress source records Amazon CloudWatch Logs** section, choose **Turn on decompression**.
 - b. In the **Transform source records with AWS Lambda** section, provide values for the following field:

Data transformation

To create a Firehose stream that doesn't transform incoming data, do not check the **Enable data transformation** checkbox.

To specify a Lambda function for Amazon Data Firehose to invoke and use to transform incoming data before delivering it, check the **Enable data transformation** checkbox. You can configure a new Lambda function using one of the Lambda blueprints or choose an existing Lambda function. Your Lambda function must contain the status model that is required by Amazon Data Firehose. For more information, see [Transform source data in Amazon Data Firehose](#).

2. In the **Convert record format** section, provide values for the following field:

Record format conversion

To create a Firehose stream that doesn't convert the format of the incoming data records, choose **Disabled**.

To convert the format of the incoming records, choose **Enabled**, then specify the output format you want. You need to specify an AWS Glue table that holds the schema that you want Amazon Data Firehose to use to convert your record format. For more information, see [Convert input data format in Amazon Data Firehose](#).

STEP 4:

Configure Destination settings:

Configure destination settings for Amazon S3:

◆ Basic Configuration

- **S3 Bucket:** Choose or create an S3 bucket where Firehose will deliver the data.
- **New Line Delimiter:**
 - **Enabled:** Adds a newline between records (recommended for Athena queries).
 - **Disabled:** No delimiter added.

◆ Data Processing Options

- **Dynamic Partitioning:**
 - **Enabled:** Allows data to be delivered to different S3 prefixes based on record content.
 - Requires **multi-record deaggregation** to be enabled if data is aggregated.
 - If your data is aggregated, dynamic partitioning can be applied only after data deaggregation is performed. So if you enable dynamic partitioning to your aggregated data, you must choose **Enabled** to enable multi record deaggregation.
 - **Multi Record Deaggregation:**
 - **Enabled:** Separates multiple records in a single API call.
 - **Type:** Choose between `JSON` or `Delimited`.
 - **Inline Parsing:**
 - **Enabled:** Extracts partitioning keys directly from data using `jq` expressions.
-

◆ Partitioning Keys & Prefixes

- **Dynamic Partitioning Keys:**
 - Specify up to **50 keys** using `jq 1.6` expressions.
- **S3 Bucket Prefix:**
 - Must match the number of partitioning keys.
 - Format:
 - Inline parsing: `partitionKeyFromQuery:keyID`
 - Lambda: `partitionKeyFromLambda:keyID`
 - You can also use: `!{namespace:value}`

◆ Additional Settings

- **Time Zone for Prefixes:** Choose a time zone for date/time in S3 object prefixes (default is UTC).
- **Buffering Hints:** Configure buffer size and interval before delivery.
- **Compression:** Options include `GZIP`, `Snappy`, `Zip`, `Hadoop-Compatible Snappy`, or `None`.
- **File Extension Format:** Optional custom file extension (starts with a dot, max 128 characters).
- **Encryption:**
 - Use default S3 encryption or AWS KMS-managed keys (`aws/s3` or customer-managed).

Configure destination settings for Amazon Redshift:

Amazon Redshift Provisioned Cluster

✅ Step-by-Step Setup for Firehose → Redshift (Provisioned Cluster)

Step 1: Prepare Redshift Cluster

1. **Ensure the cluster is publicly accessible:**
 - Go to the Redshift console.
 - Modify your cluster and enable **Publicly Accessible**.
2. **Disable Enhanced VPC Routing:**
 - In the cluster settings, ensure **Enhanced VPC Routing** is **disabled**.
 - **For serverless workgroup redshift there is no Enhanced VPC.**
3. **Security Group Configuration:**
 - Add **inbound rules** to allow Firehose IP ranges (refer to AWS IP ranges).

Step 2: Create or Choose an S3 Bucket

- Firehose will first deliver data to this bucket.

- Ensure the bucket is in the **same region** as your Redshift cluster (or specify `REGION` in `COPY` options).

Step 3: Create Redshift Table

- Create a table in your Redshift database with the appropriate schema.
- Example:

```
CREATE TABLE logs (  
  id INT,  
  message VARCHAR(255),  
  timestamp TIMESTAMP  
);
```

Step 4: Configure Firehose Delivery Stream

1. **Source:** Choose your data source (Direct PUT or Kinesis Data Stream).
2. **Destination:** Select **Amazon Redshift**.
3. **Cluster:** Choose your Redshift cluster.
4. **Authentication:**
 - Option 1: Enter **username/password**.
 - Option 2: Use **AWS Secrets Manager** to store credentials.
5. **Database & Table:** Specify the target database and table.
6. **Columns (Optional):** Map columns if S3 data has fewer fields.
7. **Intermediate S3 Bucket:** Choose the bucket created in Step 2.
8. **S3 Prefix (Optional):** Customize the prefix or leave default (`YYYY/MM/dd/HH`).
9. **COPY Options:**
 - Example: `GZIP REGION 'us-east-1'`
10. **Retry Duration:** Set retry time (e.g., 300 seconds).
11. **Buffering Hints:** Set buffer size and interval (e.g., 5MB, 300 seconds).
12. **Compression:** Choose `GZIP` or `None`.
13. **File Extension (Optional):** Set if needed (e.g., `.log`).
14. **Encryption:** Choose default or KMS-managed key.

Step 5: Grant Permissions

- Ensure the Redshift user has: `GRANT INSERT ON logs TO firehose_user;`
- Attach the necessary IAM role to Firehose with permissions for:
 - Writing to S3
 - Executing Redshift `COPY` command
 - Accessing Secrets Manager (if used)

Configure destination settings for HTTP Endpoint:

✔ Key Configuration Settings

◆ HTTP Endpoint Details

- **HTTP Endpoint Name (Optional):** A friendly name for your destination (e.g., `MyAnalyticsEndpoint`).
- **HTTP Endpoint URL:** Must be a valid **HTTPS URL** (e.g., `https://api.example.com/data`).

◆ Authentication

- **Access Key (Optional):** Manually enter the access key if required by the endpoint.
- **AWS Secrets Manager:** You can store and retrieve the access key securely from Secrets Manager.

◆ Content Encoding

- Choose between:
 - **GZIP:** Compresses the request body.
 - **Disabled:** Sends data uncompressed.

◆ Retry Logic

- **Retry Duration:** Set how long Firehose should retry sending data (0–7200 seconds).
 - Firehose waits for an **acknowledgment** from the endpoint.
 - If it fails or times out, it retries until the duration expires.
 - If retries fail, Firehose **backs up the data to S3**.

◆ Parameters (Optional)

- Add **key-value pairs** to each HTTP request for identification or routing (e.g., `env=prod, source=firehose`).

◆ Buffering Hints

- Firehose buffers data before sending.
- If you see **HTTP 413 (Payload Too Large)** errors in CloudWatch Logs:
 - **Reduce the buffer size** in Firehose settings.

- ◆ Backup to S3

- If delivery to the HTTP endpoint fails, Firehose automatically backs up the data to your configured **Amazon S3 bucket**.

Configure destination settings for MongoDB Atlas:

- ◆ API Gateway URL

- Must be a valid **HTTPS URL** in the format:

`https://xxxxx.execute-api.<region>.amazonaws.com/<stage>`

- This endpoint is typically backed by an **AWS Lambda function** that interacts with **MongoDB Atlas**.

- ◆ Authentication

- **API Key:**
 - Obtain from MongoDB Atlas or the API Gateway owner.
 - Used to authorize Firehose to send data.
- **AWS Secrets Manager:**
 - Store the API key securely.
 - Firehose can retrieve it during delivery.

- ◆ Content Encoding

- Choose:
 - **GZIP:** Compresses request body.
 - **Disabled:** Sends data uncompressed.

- ◆ Retry Duration

- Set how long Firehose should retry sending data (0–7200 seconds).
- Firehose waits for an **acknowledgment** from the endpoint.
- If it fails or times out:
 - Firehose retries until the duration expires.
 - If still unsuccessful, it **backs up the data to S3**.

◆ Buffering Hint

- Firehose buffers data before sending.
- If you see **HTTP 413 (Payload Too Large)** errors:
 - Reduce the buffer size in Firehose settings.

◆ Parameters (Optional)

- Add **key-value pairs** to each HTTP request.
- Useful for tagging, routing, or identifying data sources.

◆ Backup to S3

- If delivery to MongoDB Atlas fails, Firehose stores the data in your configured **Amazon S3 bucket**.

✅ Steps to Get the MongoDB Atlas API Gateway URL

◆ 1. Deploy a Serverless Integration

MongoDB Atlas does **not expose a direct HTTPS endpoint** for Firehose. Instead, you need to:

- Create an **AWS API Gateway** endpoint.
- Back it with an **AWS Lambda function** that connects to MongoDB Atlas.

This Lambda function will:

- Receive data from Firehose via API Gateway.
- Parse and validate the data.
- Insert it into your MongoDB Atlas cluster.

◆ 2. Get the API Gateway URL

Once you deploy the serverless application:

- Go to the **CloudFormation Outputs** section in the AWS Console.
- Look for the output named something like `ApiGatewayInvokeURL`.
- It will look like:

```
https://xxxxx.execute-api.<region>.amazonaws.com/<stage>
```

- This is the URL you enter in the **HTTP Endpoint URL** field in Firehose.

◆ 3. Get the API Key

- The deployment will also generate an **API key**.
- You can find it in the CloudFormation Outputs or Secrets Manager.
- Use this key for authentication in Firehose.

MongoDB Atlas Connection String

It looks something like this:

```
mongodb+srv://<username>:<password>@cluster0.mongodb.net/<dbname>?retryWrites=true&w=majority
```

This string contains:

- **Protocol:** `mongodb+srv` (for DNS seed list)
- **Username & Password:** Your database credentials
- **Cluster Address:** The hostname of your Atlas cluster
- **Database Name:** The specific database you want to connect to
- **Options:** Retry settings, write concern, etc.

We can find your connection string in the **MongoDB Atlas dashboard**:

✅ Steps:

1. Log in to MongoDB Atlas.
2. Go to your **Project** and select your **Cluster**.
3. Click **Connect**.
4. Choose **Connect your application**.
5. Select your **driver and version** (e.g., Node.js, Python).
6. Copy the **connection string** shown.

Need to replace `<username>`, `<password>`, and `<dbname>` with your actual values.

Steps for configure MongoDB as destination steps [MongoDB-Firehose-Ingestion GitHub project](#).

Configure destination settings for Snowflake

✅ Step-by-Step: Configure Firehose → Snowflake

◆ Step 1: Get Your Snowflake Account URL

You can find your **Snowflake account URL** in several ways:

Option 1: From the Snowflake Web UI (Snowsight)

1. Log in to Snowflake.

2. Look at the **browser address bar**.
3. The URL will look like: `https://xy12345.us-east-1.aws.snowflakecomputing.com`
 - o `xy12345` is your **account locator**.
 - o `us-east-1` is your **AWS region**.
 - o `aws` is your **cloud provider**.

Option 2: Using SQL

Run the following commands in Snowflake:

```
SELECT CURRENTREGION();
```

```
SELECT CURRENTACCOUNT();
```

These will return your region and account locator, which you can use to build the URL:

```
<account_locator>.<region>.<cloud_provider>.snowflakecomputing.com
```

Example:

```
xy12345.us-east-1.aws.snowflakecomputing.com
```

[\[1\]](#)

◆ Step 2: Prepare Snowflake

1. **Create a database, schema, and table** to receive data.
2. Ensure the **user login** has `INSERT` permissions on the target table.
3. Optionally, create a **VARIANT column** for semi-structured data and a **metadata column**.

◆ Step 3: Set Up Authentication

You can authenticate Firehose to Snowflake using:

Option 1: Manual Entry

- **User login**
- **Private key** (PKCS8 format, no headers/footers or line breaks)
- **Passphrase** (if encrypted)

Option 2: AWS Secrets Manager

- Store the credentials securely.
- Select the secret in Firehose configuration.

we have **two options** to authenticate Firehose with Snowflake:

◆ Option 1: Manual Entry (Key Pair Authentication)

1. Create a Snowflake User

- Ensure the user has `INSERT` privileges on the target table.

2. Generate a Key Pair

Use OpenSSL to generate a public/private key pair:

```
openssl genpkey -algorithm RSA -out snowflakeprivatekey.pem -pkeyopt rsakeygenbits:2048
```

```
openssl rsa -in snowflakeprivatekey.pem -pubout -out snowflakepublickey.pem
```

```
``
```

3. Convert Private Key to PKCS8 Format

```
openssl pkcs8 -topk8 -inform PEM -outform PEM -in snowflakeprivatekey.pem -out  
snowflakeprivatekey_pkcs8.pem -nocrypt
```

🔒 If you want to encrypt the private key, remove `-nocrypt` and provide a passphrase.

4. Upload Public Key to Snowflake

In Snowflake:

```
ALTER USER yourusername SET RSAPUBLICKEY='yourpublickey_content';
```

```
``
```

5. Prepare Private Key for Firehose

- Remove:
 - `-----BEGIN PRIVATE KEY-----`
 - `-----END PRIVATE KEY-----`
 - All line breaks
- Paste the cleaned key into Firehose configuration.

6. Enter Passphrase (if encrypted)

◆ Option 2: Use AWS Secrets Manager

1. Create a Secret

In AWS Secrets Manager:

- Store the following as key-value pairs:
 - `user_login`
 - `private_key` (cleaned PKCS8 format)
 - `passphrase` (if encrypted)

2. Reference the Secret in Firehose

- In Firehose destination settings, choose the secret from the dropdown.

To generate a **private key** for authenticating Amazon Firehose with **Snowflake**, follow these steps using **OpenSSL**:

✓ Step-by-Step: Generate a Private Key in PKCS8 Format

◆ 1. Generate a Private Key (PKCS#1 Format)

```
openssl genpkey -algorithm RSA -out snowflakeprivatekey.pem -pkeyopt rsakeygenbits:2048
```

This creates a private key file named `snowflake_private_key.pem`.

◆ 2. Convert to PKCS#8 Format (Required by Snowflake)

```
openssl pkcs8 -topk8 -inform PEM -outform PEM -in snowflakeprivatekey.pem -out snowflakeprivatekey_pkcs8.pem -nocrypt
```

- `-nocrypt` means the key is **not encrypted**.
- If you want to **encrypt** the key, remove `-nocrypt` and OpenSSL will prompt you for a passphrase.

◆ 3. Prepare the Key for Firehose

- Open `snowflake_private_key_pkcs8.pem` in a text editor.
- Remove:
 - `-----BEGIN PRIVATE KEY-----`
 - `-----END PRIVATE KEY-----`
 - All **line breaks**
- You should end up with a **single-line string** like:
`MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAKcwggSjAgEAAoIBAQD...`

◆ Step 4: Configure Firehose Delivery Stream

In the AWS Console:

1. Choose **Snowflake** as the destination.
 2. Enter:
 - **Snowflake account URL**
 - **Authentication method**
 - **Database, schema, and table**
 - **Content column name** (e.g., `raw_data`)
 - **Metadata column name** (optional)
 3. Choose:
 - **Default or custom Snowflake role**
 - **Public or Private connectivity**
 - For Private, provide **VPCE ID** if using AWS PrivateLink.
 4. Set:
 - **Retry duration** (0–7200 seconds)
 - **Buffering hints** (size and interval)
-

STEP 5: Testing Firehose stream with sample data:

Test with Amazon S3

Use the following procedure to test your Firehose stream with Amazon Simple Storage Service (Amazon S3) as the destination.

To test a Firehose stream using Amazon S3

1. Open the Firehose console at <https://console.aws.amazon.com/firehose/>.
2. Choose an active Firehose stream. The Firehose stream must be in **Active** status before you can start sending data.
3. Under **Test with demo data**, choose **Start sending demo data** to generate sample stock ticker data.
4. Follow the onscreen instructions to verify that data is being delivered to your S3 bucket. Note that it might take a few minutes for new objects to appear in your bucket, based on the buffering configuration of your bucket.
5. When the test is complete, choose **Stop sending demo data** to stop incurring usage charges.

Test with Amazon Redshift

Use the following procedure to test your Firehose stream with Amazon Redshift as the destination.

To test a Firehose stream using Amazon Redshift

1. Your Firehose stream expects a table to be present in your Amazon Redshift cluster. [Connect to Amazon Redshift through a SQL interface](#) and run the following statement to create a table that accepts the sample data.

```
2. create table firehose_test_table
3. (
4.   TICKER_SYMBOL varchar(4),
5.   SECTOR varchar(16),
6.   CHANGE float,
7.   PRICE float
```

```
);
```

8. Open the Firehose console at <https://console.aws.amazon.com/firehose/>.
9. Choose an active Firehose stream. The Firehose stream must be in **Active** status before you can start sending data.
10. Edit the destination details for your Firehose stream to point to the newly created `firehose_test_table` table.
11. Under **Test with demo data**, choose **Start sending demo data** to generate sample stock ticker data.
12. Follow the onscreen instructions to verify that data is being delivered to your table. Note that it might take a few minutes for new rows to appear in your table, based on the buffering configuration.
13. When the test is complete, choose **Stop sending demo data** to stop incurring usage charges.
14. Edit the destination details for your Firehose stream to point to another table.
15. (Optional) Delete the `firehose_test_table` table.

8.2 Transform source data in Amazon Data Firehose

- When **data transformation** is enabled in Firehose (via **AWS Lambda**), Firehose **buffers incoming data** before invoking the Lambda function.
- **Buffer size range:** 0.2 MB to 3 MB
 - **Default:** 1 MB for most destinations
 - **Snowflake & Splunk:** 256 KB
- **Buffer interval range:** 0 to 900 seconds
 - **Default:** 60 seconds
 - **Snowflake:** 30 seconds
- You can customize these using the `ProcessingConfiguration` parameter in the Firehose API.
- Firehose invokes Lambda **synchronously** with each buffered batch.
- Transformed data is returned to Firehose and delivered to the destination once either:
 - The **buffer size** is reached, or
 - The **buffer interval** expires.

The following parameters are required for all transformed records from Lambda.

- `recordId` – The record ID is passed from Amazon Data Firehose to Lambda during the invocation. The transformed record must contain the same record ID. Any mismatch between the ID of the original record and the ID of the transformed record is treated as a data transformation failure.
- `result` – The status of the data transformation of the record. The possible values are: `Ok` (the record was transformed successfully), `Dropped` (the record was dropped intentionally by your processing logic), and `ProcessingFailed` (the record could not be transformed). If a record has a status of `Ok` or `Dropped`, Amazon Data Firehose considers it successfully processed. Otherwise, Amazon Data Firehose considers it unsuccessfully processed.
- `data` – The transformed data payload, after base64-encoding.

Following is a sample Lambda result output:

```
{  
  "recordId": "<recordId from the Lambda input>",  
  "result": "Ok",  
}
```



```
"data": "<Base64 encoded Transformed data>"
}
```

To see the blueprints that are available in the AWS Lambda console

1. Sign in to the AWS Management Console and open the AWS Lambda console at <https://console.aws.amazon.com/lambda/>.
2. Choose **Create function**, and then choose **Use a blueprint**.
3. In the **Blueprints** field, search for the keyword `firehose` to find the Amazon Data Firehose Lambda blueprints.

List of blueprints:

- **Process records sent to Amazon Data Firehose stream (Node.js, Python)**

This blueprint shows a basic example of how to process data in your Firehose data stream using AWS Lambda.

Latest release date: November, 2016.

Release notes: none.

- **Process CloudWatch Logs sent to Firehose**

This blueprint is deprecated. Do not use this blueprint. It might incur high charges when the decompressed CloudWatch Logs data is more than 6MB (Lambda limit). For information on processing CloudWatch Logs sent to Firehose, see [Writing to Firehose Using CloudWatch Logs](#).

- **Convert Amazon Data Firehose stream records in syslog format to JSON (Node.js)**

This blueprint shows how you can convert input records in RFC3164 Syslog format to JSON.

Latest release date: Nov, 2016.

Release notes: none.

To see the blueprints that are available in the AWS Serverless Application Repository

1. Go to [AWS Serverless Application Repository](#).
2. Choose **Browse all applications**.
3. In the **Applications** field, search for the keyword `firehose`.

8.3 Partition streaming data in Amazon Data Firehose

- **Dynamic partitioning in Amazon Kinesis Data Firehose** allows streaming data to be automatically grouped and delivered to Amazon S3 using keys like `customer_id` or `transaction_id`.
- This enables **efficient, cost-effective analytics** using services like **Athena, EMR, Redshift Spectrum, QuickSight**, and **AWS Glue** for ETL.
- It **reduces the need for complex partitioning applications**, simplifies architecture, and **lowers time-to-insight**.
- Partitioning improves **query performance**, **reduces costs**, and allows **granular access control**.
- Firehose dynamically delivers data to **S3 prefixes based on keys**, making it easier to analyze and process streaming data.

◆ Purpose of Dynamic Partitioning

- Enables **targeted delivery of streaming data** to Amazon S3 by using **partitioning keys** (e.g., `customer_id`, `country`, `event_timestamp`).
- Helps in **filtering, organizing, and analyzing** data efficiently using services like Athena, EMR, Redshift Spectrum, QuickSight, and AWS Glue.

◆ Partitioning Key Methods

You must configure **at least one** of the following methods:

1. Inline Parsing (using jq)

- Works with **JSON-formatted** records.
- Uses **jq expressions** to extract keys.

```
{
  "type": {
    "device": "mobile",
    "event": "user_clicked_submit_button"
  },
  "customer_id": "1234567890",
  "event_timestamp": 1565382027,    #epoch timestamp
  "region": "sample_region"
}
```

Parameter	jq expression
customer_id	.customer_id
device	.type.device
year	.event_timestamp strftime("%Y")
month	.event_timestamp strftime("%m")
day	.event_timestamp strftime("%d")
hour	.event_timestamp strftime("%H")

2. AWS Lambda Function

- Used for **non-JSON**, **compressed**, or **encrypted** data.
- Custom code extracts partitioning keys.
- Example Python Lambda function:
 - Decodes Base64 data.
 - Parses JSON.
 - Extracts keys like `customerId`, `year`, `month`, `day`, `hour`, `minute`.
 - Returns them in `metadata.partitionKeys`.

```

from __future__ import print_function
import base64
import json
import datetime

# Signature for all Lambda functions that user must implement
def lambda_handler(firehose_records_input, context):
    print("Received records for processing from DeliveryStream: " +
firehose_records_input['deliveryStreamArn']
        + ", Region: " + firehose_records_input['region']
        + ", and InvocationId: " +
firehose_records_input['invocationId'])

    # Create return value.
    firehose_records_output = {'records': []}

    # Create result object.
    # Go through records and process them

    for firehose_record_input in firehose_records_input['records']:
        # Get user payload
        payload = base64.b64decode(firehose_record_input['data'])

```

```

        json_value = json.loads(payload)

        print("Record that was received")
        print(json_value)
        print("\n")
        # Create output Firehose record and add modified payload and
        record ID to it.
        firehose_record_output = {}
        event_timestamp =
datetime.datetime.fromtimestamp(json_value['eventTimestamp'])
        partition_keys = {"customerId": json_value['customerId'],
                           "year": event_timestamp.strftime('%Y'),
                           "month": event_timestamp.strftime('%m'),
                           "day": event_timestamp.strftime('%d'),
                           "hour": event_timestamp.strftime('%H'),
                           "minute": event_timestamp.strftime('%M')}

        # Create output Firehose record and add modified payload and
        record ID to it.
        firehose_record_output = {'recordId':
firehose_record_input['recordId'],
                                'data':
firehose_record_input['data'],
                                'result': 'Ok',
                                'metadata': { 'partitionKeys':
partition_keys }}

        # Must set proper record ID
        # Add the record to the list of output records.

firehose_records_output['records'].append(firehose_record_output)

        # At the end return processed records

    return firehose_records_output

```

◆ S3 Bucket Prefixes in Firehose

- When using **Amazon S3 as the destination**, you must specify an **S3 bucket**.
- **Prefixes** act like folders to organize data.
- With **dynamic partitioning enabled**, specifying **S3 bucket prefixes is mandatory**.

◆ Prefix Format for Dynamic Partitioning

- Prefixes are built using **partitioning keys** and follow this format: `!{namespace:keyID}`
 - namespace can be:
 - `partitionKeyFromQuery` → for **inline parsing (jq)**
 - `partitionKeyFromLambda` → for **Lambda-based extraction**
- Example prefix using inline parsing:

"Prefix": "customerid=!{partitionKeyFromQuery:customerid}/

device=!{partitionKeyFromQuery:device}/

year=!{partitionKeyFromQuery:year}/

month=!{partitionKeyFromQuery:month}/

day=!{partitionKeyFromQuery:day}/

hour=!{partitionKeyFromQuery:hour}/"

- Firehose evaluates this at runtime and delivers grouped records to: `s3://my-logs-prod/customer_id=1234567890/device=mobile/year=2019/month=08/day=09/hour=20/`
...
- You can also use **Hive-style formatting**:
`customerid=!{partitionKeyFromQuery:customerid}`

◆ New Line Delimiter Option

- You can **enable New Line Delimiter** to separate records in S3 objects.
- This is useful for:
 - Easier parsing.
 - Handling **aggregated data**, where deaggregation removes new lines.

◆ Dynamic Partitioning with Aggregated Data

- You **can apply dynamic partitioning** to aggregated data (e.g., multiple events in a single API call).
- **Requirement:** Data must be **deaggregated first**.

◆ Multi-Record Deaggregation Types

1. JSON Type

- Records are separated based on **consecutive JSON objects**.
- Valid formats:
 - `{"a":1}{ "a":2}`
 - `{"a":1}\n{"a":2}` (JSONL)
- **Invalid format:** `[{"a":1}, {"a":2}]` (JSON array)

2. Delimited Type

- Records are separated using a **custom delimiter**.
- Delimiter must be **Base64-encoded**.
 - Example: `####` → Base64 = `IyMjIw==`
- Useful for non-JSON formats or custom structured data.

- **Limit:** Max **500 records per aggregated record**.

◆ Firehose Behavior

- When dynamic partitioning is enabled:
 - Firehose **parses each record** using the selected deaggregation method.
 - Then applies **partitioning logic** based on extracted fields.
- • Uses **Amazon S3** as the destination.
- • Enables **dynamic partitioning**.
- • Applies **multi-record deaggregation** (JSON or Delimited).
- • Uses **inline parsing** or **Lambda** for partitioning keys.
- • Specifies a **custom S3 prefix** using partitioning expressions.

8.4 Understand data delivery

◆ Request Format (Firehose → Endpoint)

- Only **HTTPS on port 443** is supported.
- Firehose sends **JSON-formatted requests** with headers like:
 - `X-Amz-Firehose-Request-Id`: unique ID for tracking.
 - `X-Amz-Firehose-Protocol-Version`: always `1.0`.
 - `X-Amz-Firehose-Access-Key`: credentials or token.
 - `X-Amz-Firehose-Common-Attributes`: optional metadata.
- Request body includes:
 - `requestId`, `timestamp`, and an array of `records` (Base64-encoded).
- Max body size: **64 MiB** before compression.
- Optional GZIP compression.

```
{
  "requestId": "ed4acda5-034f-9f42-bba1-f29aea6d7d8f",
  "timestamp": 1578090901599
  "records": [
    {
      "data": "aGVsbG8="
    },
    {
      "data": "aGVsbG8gd29ybGQ="
    }
  ]
}
```

◆ Response Format (Endpoint → Firehose)

- Must return **HTTP 200** for success.
- Only `application/json` content type is accepted.
- No compression allowed in response.
- Response body includes:
 - `requestId`, `timestamp`, and optional `errorMessage`.
- Max response size: **1 MiB**.
- Firehose does **not follow redirects (3XX)**.
- Status code 413 is treated as a **permanent failure**.

Failure **Case** (HTTP Response Code **4xx** or **5xx**)

```
{
  "requestId": "ed4acda5-034f-9f42-bba1-f29aea6d7d8f",
  "timestamp": "1578090903599",
  "errorMessage": "Unable to deliver records due to unknown error."
}
```

Success **case** (HTTP Response Code **200**)

```
{
  "requestId": "ed4acda5-034f-9f42-bba1-f29aea6d7d8f",
  "timestamp": 1578090903599
}
```

9. Deliver data to Apache Iceberg Tables with Amazon Data Firehose

◆ Apache Iceberg Overview

- A **high-performance open-source table format** for big data analytics.
- Supports **SQL-like operations** on data lakes in Amazon S3.

- Compatible with engines like **Spark, Flink, Trino, Hive, and Impala**.

◆ Firehose Integration with Iceberg

- Firehose can **stream data directly into Apache Iceberg tables** stored in Amazon S3.
- Supports both:
 - **Self-managed Iceberg tables**: You handle compaction, snapshot expiration, etc.
 - **Amazon S3 Tables**: AWS-managed, optimized for performance and cost.

◆ Advanced Features

- **Route records to multiple Iceberg tables** from a single Firehose stream.
- Supports **insert, update, and delete operations** on Iceberg tables.
- Integrates with **AWS Lake Formation** for:
 - **Fine-grained access control** (table-level and column-level permissions).
 - Centralized permission management.

◆ 1. Create an Amazon S3 Bucket

This bucket will store:

- **Iceberg table metadata** (like snapshots, manifests).
- **Actual data files** (in Parquet format).

✓ Steps:

- Go to the **Amazon S3 Console**.
- Click **Create bucket**.
- Choose a globally unique name (e.g., `my-iceberg-data-lake`).
- Select region and configure settings (versioning, encryption, etc.).
- Click **Create bucket**.

✦ **Note:** This bucket path will be referenced when creating Iceberg tables and configuring Firehose.

◆ 2. Create an IAM Role with Required Permissions

This role allows Firehose to:

- Access **Glue Data Catalog**.
- Write data to **Amazon S3**.
- (Optional) Access **Lake Formation** for fine-grained permissions.

✓ Required Permissions:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "glue:GetTable",
        "glue:GetDatabase",
        "glue:UpdateTable",
        "s3:PutObject",
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": "*"
    }
  ]
}
```

✦ If using **Lake Formation**, also grant access via Lake Formation console.

◆ 3. Create Apache Iceberg Tables

You must create Iceberg tables **before** creating the Firehose stream **if** you plan to use:

- **Unique keys** for update/delete operations.

Otherwise, table creation can be deferred.

To create an Iceberg table

1. Sign in to the AWS Management Console and open the AWS Glue console at <https://console.aws.amazon.com/glue/>.
2. Under Data Catalog, choose **Tables**, and use the **Create table** button to specify the following attributes:

- **Table name** – Enter a name for the table. If you're using Athena to access tables, use these [naming tips](#) in the Amazon Athena User Guide.
- **Database** – Choose an existing database or create a new one.
- **Description** – The description of the table. You can write a description to help you understand the contents of the table.
- **Table format** – For **Table format**, choose Apache Iceberg.
- **Enable compaction** – Choose **Enable compaction** to compact small Amazon S3 objects in the table into larger objects.
- **IAM role** – To run compaction, the service assumes an IAM role on your behalf. You can choose an IAM role using the drop-down. Ensure that the role has the permissions required to enable compaction.

To learn more about the required permissions, see [Table optimization prerequisites](#).

- **Location** – Specify the path to the folder in Amazon S3 that stores the metadata table. Iceberg needs a metadata file and location in the Data Catalog to be able to perform reads and writes.
- **Schema** – Choose **Add columns** to add columns and data types of the columns. You have the option to create an empty table and update the schema later.

✓ Table Format Requirements:

- **Table format version:** Must be **V2** (V1 is not supported).
- **Storage format:** Firehose writes data in **Parquet**.
- **Write mode:** Firehose uses **Merge-on-Read (MOR)** for row-level operations.

✓ Creation Options:

- Use **AWS Glue Studio**, **Athena**, or **Spark/Flink** to create the table.
- Register the table in **AWS Glue Data Catalog**.

✦ Firehose will validate the existence of the table and keys during stream creation.

◆ 4. (Optional) Prerequisites for Amazon S3 Tables

If you're using **Amazon S3 Tables** (AWS-managed Iceberg tables):

✓ Additional Setup:

- Create:
 - **S3 Table bucket**
 - **Namespace**

- **Tables** within the bucket
- Follow steps in **Getting started with Amazon S3 Tables**.

✓ Column Naming:

- All column names must be **lowercase** due to catalog integration limitations.

✓ IAM Role:

- Same role used for Firehose must have permissions in **Lake Formation**.

◆ 5. Configure AWS Lake Formation Permissions (Optional)

Lake Formation provides:

- **Table-level and column-level access control.**
- Centralized permission management for Glue resources.

✓ Steps:

- Go to **Lake Formation Console**.
- Grant permissions to the IAM role used by Firehose.
- Assign access to:
 - Specific databases/tables
 - Columns (if needed)

For step-by-step integration, [Build a data lake for streaming data with Amazon S3 Tables and Amazon Data Firehose](#).

10. Amazon Kinesis Data Analytics Using Apache Flink Zepplin Notebook

STEPS :

[Migrating to Apache Flink Zepplin](#)

Steps:

1. We need to create a stream for produce the data.
2. Then, we after creating the stream click the stream and choose processes near Action and then click Process in real time.
3. Then, Choose Apache Flink -Studio NoteBook.
4. Then Give name and then it asks AWS Glue DataBase for store the Metadata and source and table.
5. So we need to create database and then we need to choose.
6. Create Table in NoteBook by run this statement.

```
CREATE TABLE KinesisTable (
  `user_id` BIGINT,
  `item_id` BIGINT,
  `category_id` BIGINT,
  `behavior` STRING,
  `ts` TIMESTAMP(3)
)
PARTITIONED BY (user_id, item_id)
WITH (
  'connector' = 'kinesis',
  'stream.arn' = 'arn:aws:kinesis:us-east-1:012345678901:stream/my-
streamname',
  'aws.region' = 'us-east-1',
  'source.init.position' = 'LATEST',
  'format' = 'csv'
);
```

7. Create another Stream for consuming or create dynamodb table also for Consuming Records.
8. Create another table for to hold the data by connecting use sql commands in connector give either kinesis-output-stream or dynamodb.
9. Then use insert command select some records and perform operations and insert into another table.
10. Then the data will be appear on that table.
11. We perform Sql operation from source data by running queries from that source table in that NoteBook.

Flink SQL is similar to standard SQL, but it has some **important differences** because it's designed for **streaming data** rather than static datasets.

Key Differences Between Flink SQL and Traditional SQL

Feature	Flink SQL	Traditional SQL
Data Type	Streams (real-time)	Tables (static)
Time Handling	Event time, processing time, watermarks	Usually no concept of time
Windowing	Supports tumbling, sliding, session windows	Not applicable
Joins	Stream-stream joins with time constraints	Table joins
DDL	Uses <code>CREATE TABLE ... WITH (...)</code> for connectors	Uses <code>CREATE TABLE ...</code> with schema only
Connectors	Kinesis, Kafka, S3, JDBC, etc.	Usually local or cloud databases
Watermarks	Required for time-based operations	Not used

10.1 Introduction to Flink SQL

◆ What is Flink SQL?

Flink SQL is a powerful query language built on top of **Apache Flink**, designed to process **streaming** and **batch** data using SQL syntax.

It allows developers and analysts to write SQL queries to analyze **real-time data streams** (like from Kinesis, Kafka, etc.) without writing complex Java or Scala code.

Example Use Case

Imagine you're tracking user actions on a website in real time. Each action (click, view, purchase) is sent to a **Kinesis stream**.

With Flink SQL, you can write a query like:

```
SELECT
  userid,
  COUNT(*) AS actioncount,
  TUMBLESTART(eventtime, INTERVAL '1' MINUTE) AS windowstart
FROM useractionsstream
GROUP BY
  userid,
  TUMBLE(event_time, INTERVAL '1' MINUTE);
```

This query counts how many actions each user performs **every minute**, in real time.

◆ Differences from Traditional SQL

Feature	Flink SQL	Traditional SQL
Data Type	Streams (live data)	Tables (static data)
Time Awareness	Event time, watermarks	Usually no time semantics
Windowing	TUMBLE, HOP, SESSION	Not available
Continuous Execution	Runs indefinitely	Runs once and stops
Connectors	Kinesis, Kafka, S3, etc.	Mostly databases

10.2 Streaming vs Batch Processing in Flink SQL

◆ 1. Batch Processing

- Processes **static datasets** (e.g., files, databases)
- Query runs **once**, produces a result, and **stops**
- Common in traditional SQL systems like MySQL, PostgreSQL, etc.

 Example:

You have a CSV file of user purchases:

```
userid,purchaseamount
```

```
101,200
```

102,150

101,300

A batch SQL query:

```
SELECT userid, SUM(purchaseamount) AS totalspent
FROM purchases
GROUP BY userid;
```

This runs once and gives:

101 | 500

102 | 150

♦ 2. Streaming Processing

- Processes **live data** (e.g., from Kinesis, Kafka)
- Query runs **continuously**, updating results as new data arrives
- Ideal for real-time dashboards, alerts, and analytics

 Example:

You have a Kinesis stream of user actions:

```
{"userid": "101", "action": "click", "eventtime": "2025-10-25T10:00:00Z"}
```

```
{"userid": "102", "action": "view", "eventtime": "2025-10-25T10:00:05Z"}
```

```
{"userid": "101", "action": "purchase", "eventtime": "2025-10-25T10:00:10Z"}
```

A Flink SQL streaming query:

```
SELECT
  userid,
  COUNT(*) AS actioncount,
  TUMBLESTART(eventtime, INTERVAL '1' MINUTE) AS windowstart
FROM useractions
GROUP BY
  userid,
  TUMBLE(eventtime, INTERVAL '1' MINUTE);
```

This query keeps running and updates the count **every minute** as new actions arrive.

Dynamic Tables vs Static Tables

Concept	Static Table	Dynamic Table
Definition	Fixed dataset	Continuously updated
Used in	Batch SQL	Streaming SQL
Behavior	Doesn't change	Changes with incoming data
Example	CSV file	Kinesis stream

In Flink, **streams are treated as dynamic tables**, and SQL queries operate on them as if they were regular tables — but they **update in real time**.

Great! Let's explore the concept of **Dynamic Tables in Flink SQL**, especially focusing on **append-only**, **update**, and **retract** streams.

Dynamic Tables in Flink SQL

In **Flink SQL**, a **stream** is treated as a **dynamic table** — a table that **continuously changes** as new data arrives.

Unlike traditional static tables (which are fixed), dynamic tables:

- Grow over time
- Can be updated or corrected
- Reflect the current state of a stream

◆ Types of Stream Updates

Flink SQL supports **three types of dynamic table updates**:

1. Append-Only Streams

- Only **new rows** are added.
- No updates or deletions.
- Common in simple event logs or sensor data.

Example:

```
{"userid": "101", "action": "click"}
```



```
{"userid": "102", "action": "view"}
```

Each event is added to the table — no changes to previous rows.

2. Update Streams

- Existing rows are **updated** based on keys.
- Used in aggregations like `COUNT`, `SUM`, etc.

Example:

```
SELECT userid, COUNT(*) AS actioncount
FROM useractions
GROUP BY userid;
```

As new actions arrive:

- `user_id = 101` might go from `action_count = 3` → 4
 - The table updates the existing row
-

3. Retract Streams

- Rows can be **removed** and **replaced**.
- Used when Flink cannot guarantee in-place updates (e.g., complex joins or aggregations).

 **Example:** In a join query, if one side of the join changes, Flink may retract the old result and emit a new one.

```
SELECT a.userid, b.region
FROM useractions a
JOIN userprofiles b
ON a.userid = b.user_id;
```

If `user_profiles` changes, Flink retracts the old join result and emits a new one.

Real-World Analogy

- **Append-only:** Like adding new rows to a spreadsheet.
- **Update:** Like editing a cell when new info comes in.
- **Retract:** Like deleting a row and replacing it with a corrected one.

10.3 Time Semantics in Flink SQL

Flink supports **three types of time** to help you process events accurately in real-time systems:

◆ 1. Event Time

- **Definition:** The time when the event actually occurred (e.g., when a user clicked a button).
- **Why it's important:** It allows accurate analysis even if events arrive late or out of order.
- **Used with:** `TIMESTAMP` fields and **watermarks**.

 Example:

```
{"userid": "101", "action": "click", "eventtime": "2025-10-25T10:00:00Z"}
```

Flink uses `event_time` to group this into the correct time window, even if it arrives 10 seconds late.

```
WATERMARK FOR eventtime AS eventtime - INTERVAL '5' SECOND
```

◆ 2. Processing Time

- **Definition:** The time when Flink **receives** and processes the event.
- **Fastest**, but **less accurate** for time-based analytics.
- **No need for watermarks.**

 Example:

If the above event arrives at `10:00:10`, Flink processes it at that time — regardless of the original event time.

```
CREATE TABLE actions (  
  userid STRING,  
  action STRING,  
  proctime AS PROCTIME()  
)
```

◆ 3. Ingestion Time (less commonly used)

- **Definition:** The time when the event **enters Flink**, before processing.

- Acts as a middle ground between event time and processing time.
- Automatically assigned if no time field is specified.

10.4 Watermarks in Flink SQL

◆ Purpose of Watermarks

In real-time data streams, events often arrive **out of order** or **late** due to network delays, retries, or buffering.

Watermarks help Flink decide:

- When to **close a time window**
- When to **process** and **emit results**
- How much **lateness** to tolerate

◆ How Watermarks Work

A **watermark** is a timestamp that tells Flink:

"I've seen all events up to this time — you can safely process windows before this."

It's like a moving marker that advances as newer events arrive.

Example Scenario

Imagine you're counting user clicks every minute using **event time**.

Some events arrive late:

```
{"userid": "101", "eventtime": "10:00:30"} ← arrives on time
```

```
{"userid": "102", "eventtime": "10:00:45"} ← arrives late at 10:01:10
```

```
..
```

Without watermarks, Flink might miss the late event or misplace it in the wrong window.

With watermarks:

```
WATERMARK FOR eventtime AS eventtime - INTERVAL '5' SECOND
```

Flink waits **5 seconds** beyond the event time before closing the window — allowing late events to be included.

Handling Late/Out-of-Order Data

Strategy	Description
Watermark delay	Allows late events within a time buffer
Allowed lateness	Configures how long Flink waits before finalizing results
Side outputs	Late events beyond watermark can be redirected for separate handling

Summary

Concept	Purpose
Watermark	Tells Flink when it's safe to process time windows
Late event	Arrives after its expected window
Out-of-order event	Arrives before or after other events with earlier timestamps

10.5 Windowing Functions in Flink SQL — Real-Time Examples

Imagine you're analyzing **user actions** (clicks, views, purchases) from a **Kinesis stream**. Each event includes:

```
{  
  "userid": "101",  
  "action": "click",  
  "eventtime": "2025-10-25T10:00:30Z"  
}
```

1. Tumbling Windows

Fixed-size, non-overlapping windows

Each event belongs to **exactly one window**.

Use Case:

Count user actions **every 1 minute**.

 Query:

```
SELECT
  userid,
  COUNT(*) AS actioncount,
  TUMBLESTART(eventtime, INTERVAL '1' MINUTE) AS windowstart
FROM useractions
GROUP BY
  userid,
  TUMBLE(eventtime, INTERVAL '1' MINUTE);
```

 Example:

- Events at 10:00:10, 10:00:45 → grouped into 10:00–10:01 window
- Events at 10:01:05 → grouped into 10:01–10:02 window

♦ 2. Hopping Windows

Fixed-size windows that overlap

Events can belong to **multiple windows**.

 Use Case:

Track user activity in **15-minute windows**, starting every **5 minutes**.

 Query:

```
SELECT
  userid,
  COUNT(*) AS actioncount,
  HOPSTART(eventtime, INTERVAL '5' MINUTE, INTERVAL '15' MINUTE) AS windowstart
FROM useractions
GROUP BY
  userid,
  HOP(eventtime, INTERVAL '5' MINUTE, INTERVAL '15' MINUTE);
```

 Example:

- Event at 10:07:00 appears in:
 - 10:00–10:15
 - 10:05–10:20
 - 10:10–10:25

♦ 3. Session Windows

Windows based on user inactivity

- Based on **inactivity gaps**.
- Window closes when no event arrives for a specified time.

📌 Use Case:

Group user actions into sessions — if no action for **10 minutes**, start a new session.

💬 Query:

```
SELECT
  userid,
  COUNT(*) AS actioncount,
  SESSIONSTART(eventtime, INTERVAL '10' MINUTE) AS sessionstart
FROM useractions
GROUP BY
  userid,
  SESSION(eventtime, INTERVAL '10' MINUTE);
``
```

🕒 Example:

- User clicks at 10:00, 10:05 → same session
- Next click at 10:20 → new session starts

♦ 4. Sliding Windows (via HOP in SQL)

Windows that slide continuously

📌 Use Case:

Calculate a **moving average** of actions every **minute**, over a **5-minute window**.

 Query:

```
SELECT
  userid,
  COUNT(*) AS actioncount,
  HOPSTART(eventtime, INTERVAL '1' MINUTE, INTERVAL '5' MINUTE) AS windowstart
FROM useractions
GROUP BY
  userid,
  HOP(eventtime, INTERVAL '1' MINUTE, INTERVAL '5' MINUTE);
```

 Example:

- Event at 10:03:00 appears in:
 - 10:00–10:05
 - 10:01–10:06
 - 10:02–10:07
 - 10:03–10:08
 - 10:04–10:09

♦ 5. Staggered Windows (*Advanced — Table API only*)

Used to reduce skew in parallel processing

 Use Case:

If many events fall into the same window (e.g., midnight), staggered windows help distribute load.

 Note:

Not available in Flink SQL directly — used in Java/Scala APIs with custom logic.

 Summary Table

Window Type	Behavior	Real-Time Use Case
Tumbling	Fixed, non-overlapping	Count actions per minute
Hopping	Overlapping, fixed size	Rolling 15-min activity windows

Window Type	Behavior	Real-Time Use Case
Session	Based on inactivity	User session tracking
Sliding (via HOP)	Continuous sliding	Moving averages
Staggered	Load-balanced windows	High-volume event distribution

10.6 Table Definitions (DDL) in Flink SQL

In Flink SQL, you use `CREATE TABLE ... WITH (...)` to define:

- The **schema** of your data
- The **connector** (e.g., Kinesis, Kafka, S3)
- The **format** (e.g., JSON, CSV, Avro)

◆ 1. Basic Structure

```
CREATE TABLE tablename (
  column1 TYPE,
  column2 TYPE,
  ...
) WITH (
  'connector' = 'sourceor_sink',
  'property1' = 'value1',
  ...
);
```

✅ A. Kinesis Connector

Used to read from or write to an AWS Kinesis stream.

📌 Use Case:

Read user actions from a Kinesis stream.

```
CREATE TABLE useractions (  
  userid STRING,  
  action STRING,  
  eventtime TIMESTAMP(3),  
  WATERMARK FOR eventtime AS event_time - INTERVAL '5' SECOND  
) WITH (  
  'connector' = 'kinesis',  
  'stream' = 'user-actions-stream',  
  'aws.region' = 'us-east-1',  
  'format' = 'json',  
  'scan.stream.initpos' = 'LATEST'  
);
```

✅ B. Kafka Connector

Used for Apache Kafka topics.

🔴 Use Case:

Read product views from a Kafka topic.

```
CREATE TABLE productviews (  
  productid STRING,  
  viewtime TIMESTAMP(3),  
  WATERMARK FOR viewtime AS view_time - INTERVAL '10' SECOND  
) WITH (  
  'connector' = 'kafka',  
  'topic' = 'product-views',  
  'properties.bootstrap.servers' = 'localhost:9092',  
  'format' = 'json',  
  'scan.startup.mode' = 'latest-offset'  
);
```

✅ C. S3 Connector (Filesystem)

Used to write results to S3 in batch or streaming mode.

📌 Use Case:

Write aggregated results to S3.

```
CREATE TABLE s3results (  
  userid STRING,  
  actioncount BIGINT,  
  windowstart TIMESTAMP(3)  
) WITH (  
  'connector' = 'filesystem',  
  'path' = 's3://your-bucket/results/',  
  'format' = 'csv'  
);
```

✅ D. JDBC Connector

Used to read/write from relational databases like MySQL, PostgreSQL.

📌 Use Case:

Write results to a MySQL table.

```
CREATE TABLE usersummary (  
  userid STRING,  
  actioncount BIGINT  
) WITH (  
  'connector' = 'jdbc',  
  'url' = 'jdbc:mysql://your-db-host:3306/dbname',  
  'table-name' = 'usersummary',  
  'username' = 'admin',  
  'password' = 'yourpassword'
```

);

Formats — Real-Time Examples

Format	Description	Use Case
JSON	Structured text	Most common for events
CSV	Comma-separated values	Simple logs or exports
Avro	Binary format	Efficient for large data
Debezium	CDC format	Change Data Capture from DBs

```
'scan.startup.mode' = 'latest-file' | 'earliest-file'
```

Controls which file to start reading from in a directory for s3.

 `scan.stream.initpos` in Kinesis Connector

This property tells Flink **where to start reading** from the Kinesis stream when the job begins.

 Supported Values:

Value	Description
'LATEST'	Start from the most recent record (new data only)
'TRIM_HORIZON'	Start from the oldest available record (historical + new data)

10.7 Streaming Joins in Flink SQL

In streaming systems, joins are more complex than in batch SQL because data arrives **continuously** and may be **out-of-order**. Flink SQL supports several types of joins:

1. Time-Bound Joins (Stream-to-Stream)

These joins match records from two streams **within a time window**.

 Use Case:

Join user actions with product views that happened within 5 minutes of each other.

 Example:

```
SELECT
  a.userid,
  a.action,
  b.productid
FROM useractions a
JOIN productviews b
ON a.userid = b.userid
AND a.eventtime BETWEEN b.eventtime - INTERVAL '5' MINUTE AND b.event_time + INTERVAL '5' MINUTE;
```

This ensures only events that occurred close together in time are joined.

♦ 2. Stream-to-Static Table Joins (Dimension Table Join)

Used to enrich streaming data with static reference data (e.g., user profiles, product info).

 Use Case:

Add user region info from a static table to each incoming action.

 Example:

```
SELECT
  a.userid,
  a.action,
  b.region
FROM useractions a
JOIN userprofiles FOR SYSTEMTIME AS OF a.eventtime AS b
ON a.userid = b.user_id;
```

This is a **temporal join** — it uses the event time to fetch the correct version of the static data.

 Real-Time Analogy

- **Stream-to-stream join:** Like matching chat messages from two users that happened within 5 minutes.
- **Stream-to-static join:** Like tagging each message with the user's country from a static profile list.

10.8 Aggregations in Flink SQL

♦ What Are Aggregations?

Aggregations are operations that **summarize data** — like counting events, calculating averages, or summing values.

In **streaming SQL**, aggregations are typically done **within time windows**, because the data is continuously flowing and never "complete."

♦ Common Aggregation Functions

- `COUNT (*)`: Number of records
- `SUM(column)`: Total value
- `AVG(column)`: Average value
- `MAX(column)`, `MIN(column)`: Highest/lowest values

Windowing with GROUP BY

Since streaming data is infinite, we use **windows** to group data into **finite chunks of time**.

✅ Example: Tumbling Window Aggregation

SELECT

userid,

COUNT() AS actioncount,*

TUMBLESTART(eventtime, INTERVAL '1' MINUTE) AS windowstart

FROM useractions

GROUP BY

userid,

TUMBLE(eventtime, INTERVAL '1' MINUTE);

What This Does:

- Groups events by `user_id`
- Counts how many actions each user performed
- In **1-minute intervals** (tumbling windows)

Types of Windows

Window Type	Description	Use Case
TUMBLE	Fixed-size, non-overlapping	Minute-by-minute summaries
HOP	Overlapping windows	Sliding averages
SESSION	Based on inactivity gaps	User sessions or bursts of activity

Real-World Analogy

Imagine you're counting how many people enter a store:

- **Batch SQL:** Count at the end of the day.
- **Streaming SQL with windows:** Count every minute, hour, or session — live.

10.9 Sinks and Outputs in Flink SQL

A **sink** is the destination where Flink writes the results of your streaming query. Common sinks include:

- **Amazon S3** (for storage)
- **Amazon Redshift** (for analytics)
- **Elasticsearch** (for search and dashboards)
- **Kafka / Kinesis** (for further streaming)
- **Databases** (via JDBC)

♦ Write to a Sink: `INSERT INTO`

After defining a source and processing the data, you use:

```
INSERT INTO sinktable
```

```
SELECT ... FROM sourcetable;
```

This continuously writes results to the sink as new data is processed.

✅ Example: Writing Aggregated Results to S3

1. Define the Sink Table

```
CREATE TABLE s3sink (  
  userid STRING,  
  actioncount BIGINT,  
  windowstart TIMESTAMP(3)  
) WITH (  
  'connector' = 'filesystem',  
  'path' = 's3://your-bucket/flink-results/',  
  'format' = 'csv'  
);
```

2. Insert Data into Sink

```
INSERT INTO s3sink  
SELECT  
  userid,  
  COUNT(*) AS actioncount,  
  TUMBLESTART(eventtime, INTERVAL '1' MINUTE) AS windowstart  
FROM useractions  
GROUP BY  
  userid,  
  TUMBLE(event_time, INTERVAL '1' MINUTE);
```

🧠 Other Sink Examples

◆ Redshift Sink (via JDBC)

```
CREATE TABLE redshiftsink (  
  userid STRING,  
  totalspent DOUBLE  
) WITH (  
  'connector' = 'jdbc',  
  'url' = 'jdbc:redshift://your-cluster-url',
```

```

    'table-name' = 'usersummary',
    'username' = 'admin',
    'password' = 'your-password'
);

♦ Elasticsearch Sink

CREATE TABLE essink (
    userid STRING,
    action_count BIGINT
) WITH (
    'connector' = 'elasticsearch-7',
    'hosts' = 'http://your-es-host:9200',
    'index' = 'user-actions',
    'format' = 'json'
);

```

10.10 Fault Tolerance & State Management in Flink SQL

In streaming systems, data is continuously flowing. If something fails (e.g., a node crashes), you don't want to lose data or produce incorrect results. Flink provides mechanisms to **recover gracefully** and **maintain consistency**.

✓ 1. Checkpoints

- Flink periodically saves the **state of your application** to durable storage (like S3).
- If a failure occurs, Flink can **restart from the last checkpoint**.
- This ensures **exactly-once processing** (no duplicates, no missing data).

 **Example:** If you're counting user clicks and the system crashes, Flink will resume from the last checkpoint and continue counting without losing progress.

✓ 2. Exactly-Once Semantics

- Guarantees that **each event is processed only once**, even in case of retries or failures.
- This is critical for financial transactions, billing systems, or any system where **accuracy matters**.

 **Example:** If a user makes a payment, you want to ensure it's **counted once**, not twice due to a retry.

✓ 3. Savepoints and Recovery

- **Savepoints** are manually triggered snapshots of the application state.
- Useful for:
 - Upgrading applications
 - Migrating jobs
 - Controlled restarts

 **Example:** Before deploying a new version of your Flink SQL job, you take a savepoint. If something goes wrong, you can roll back to that exact state.

How It Works in Flink SQL

- You don't need to write code for checkpoints — Flink handles it automatically.
- You can configure:
 - **Checkpoint interval**
 - **Storage location** (e.g., S3)
 - **Retention policy**

What You Might Still Need to Configure

While defaults are good for development, in **production** you may want to:

Customize:

- **Checkpoint interval** (e.g., every 1 minute vs 5 minutes)
- **Checkpoint storage location** (e.g., S3 bucket)
- **Retention policy** (keep checkpoints after job cancellation?)
- **Savepoints** (manually trigger before upgrades or deployments)
- **Parallelism and scaling** (to handle more data)

 **Example:** Custom Checkpoint Settings (in Flink YAML or Studio config)

```
state.backend: filesystem
state.checkpoints.dir: s3://your-bucket/flink-checkpoints/
execution.checkpointing.interval: 60000 # every 60 seconds
execution.checkpointing.mode: EXACTLY_ONCE
```

10.11 User-Defined Functions (UDFs) in Flink SQL

A **User-Defined Function (UDF)** is a custom function you write to perform operations that **aren't supported by built-in SQL functions**.

Flink allows you to write UDFs in:

- **Java**
- **Scala**
- **Python** (via PyFlink)

Once defined, you can use them in your SQL queries just like built-in functions.

Example Scenario

Let's say you want to **mask email addresses** in a stream for privacy.

- ♦ Without UDF:

You'd struggle to do this with plain SQL.

- ♦ With UDF:

pip install apache-flink

```
from pyflink.table import EnvironmentSettings, TableEnvironment
from pyflink.table.udf import udf
```

```
# Set up the Table Environment
env_settings = EnvironmentSettings.in_streaming_mode()
table_env = TableEnvironment.create(env_settings)
```

```
# Define a UDF
@udf(result_type="STRING")
def mask_email(email: str) -> str:
    return email.split('@')[0] + '@****.com'
```

```
# Register the UDF
table_env.create_temporary_system_function("mask_email", mask_email)
```

You write a function like:

```
def mask_email(email: str) -> str:
```

```
return email.split('@')[0] + '@****.com'
```

Then register it in Flink:

```
tableenv.createTemporarySystemFunction("maskemail", maskemail)
```

And use it in SQL:

```
SELECT userid, maskemail(email) AS maskedemail FROM userstream;
```

10.12 Pattern Detection with Flink SQL (CEP)

Complex Event Processing (CEP) is used to detect **meaningful patterns** in streams of events — like fraud detection, user behavior tracking, or system anomalies.

Flink SQL supports **pattern detection** using the **MATCH_RECOGNIZE** clause, which allows you to define **event patterns** using SQL syntax.

✓ Example Use Case: Detecting Suspicious Login Behavior

Imagine a stream of login events:

```
{"userid": "101", "status": "failed", "eventtime": "10:00:00"}
{"userid": "101", "status": "failed", "eventtime": "10:00:05"}
{"userid": "101", "status": "success", "eventtime": "10:00:10"}
```

You want to detect a pattern like:

Two failed logins followed by a successful login within 1 minute.

◆ Flink SQL Pattern Detection Example

```
SELECT *
FROM loginevents
MATCHRECOGNIZE (
  PARTITION BY userid
  ORDER BY eventtime
  MEASURES
```

```

A.eventtime AS firstfail,
B.eventtime AS secondfail,
C.eventtime AS successtime
ONE ROW PER MATCH
PATTERN (A B C)
DEFINE
  A AS A.status = 'failed',
  B AS B.status = 'failed',
  C AS C.status = 'success'
);

```

- **PARTITION BY:** Groups events by user
- **ORDER BY:** Sorts events by time
- **PATTERN (A B C):** Looks for three events in sequence
- **DEFINE:** Specifies conditions for each event
- **MEASURES:** Extracts matched event details

Real-World Applications

- **Fraud detection:** Unusual transaction patterns
 - **Security monitoring:** Failed logins, access violations
 - **User behavior:** Cart abandonment, purchase funnels
 - **IoT:** Sensor anomalies, threshold breaches
-

Let's explore **SQL over Change Data Capture (CDC)** in Flink — a powerful technique for **real-time updates from databases**.

10.13 SQL over Change Data Capture (CDC)

Change Data Capture (CDC) is a method to **track and stream changes** (INSERTs, UPDATEs, DELETEs) from a database in real time.

Instead of querying the database repeatedly, CDC lets Flink **react instantly** to changes — making it ideal for:

- Real-time dashboards
- Replication

- Event-driven architectures
-

Flink can consume CDC streams using **connectors** like:

- **Debezium** (for MySQL, PostgreSQL, SQL Server, Oracle)
- **Flink CDC connectors** (open-source)
- **Kafka or Kinesis** (if CDC events are published there)

Once connected, Flink treats the CDC stream as a **dynamic table**, and you can run SQL queries on it.

♦ Example: Real-Time Inventory Updates

Imagine a MySQL table:

```
CREATE TABLE inventory (  
  productid INT,  
  stock INT,  
  lastupdated TIMESTAMP  
);
```

Using CDC, Flink can detect changes like:

```
UPDATE inventory SET stock = 50 WHERE product_id = 101;
```

And reflect that change in a Flink SQL query:

```
SELECT productid, stock FROM inventorycdc_stream;
```

This query will **automatically update** when the stock changes — no need to re-run it.

CDC like a **live news feed** from your database:

- Every change is broadcast
- Flink listens and updates your views or outputs instantly

10.14 CloudWatch Integration with Apache Flink

✓ 1. Logging Setup

You can configure your Flink application to send logs to **CloudWatch Logs**:

- **Console Setup:**
 - AWS automatically creates a log group `/aws/kinesis-analytics/<ApplicationName>` and a log stream `kinesis-analytics-log-stream`.
 - Permissions are added to allow writing logs.
- **CLI Setup:**
 - Manually create log group and stream.
 - Use `CreateApplication` or `AddApplicationCloudWatchLoggingOption` actions.
 - Update IAM policies to allow log writing.

■ [Detailed AWS Docs – Logging Setup \[docs.aws.amazon.com\]](https://docs.aws.amazon.com/elasticmapreduce/latest/userguide/telemetry-logging.html)